

Министерство образования и науки Российской Федерации
Московский физико-технический институт
(национальный исследовательский университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра инфокоммуникационных систем и сетей

Выпускная квалификационная работа
(магистерская диссертация)

Применение методов формальной верификации для анализа правил фильтрации сетевого трафика

Автор:

Студент М01-406г группы
Дурнов Алексей Николаевич

Научный руководитель:

канд. физ.-мат. наук, доцент
Ефанов Николай Николаевич

Консультант:

Ларин Дмитрий Викторович



Москва 2026

Аннотация

Применение методов формальной верификации для анализа правил фильтрации сетевого трафика

Дурнов Алексей Николаевич

Данная работа посвящена разработке системы автоматического анализа и верификации правил фильтрации сетевого трафика для выявления ошибок конфигурации. В ходе работы был проведён обзор существующих методов анализа правил фильтрации; установлено, что известные подходы рассчитаны под конкретную платформу или ограниченный набор полей и не покрывают всё многообразие конфигураций современных межсетевых экранов. Для устранения этого ограничения была разработана унифицированная вендору-независимая модель представления правил фильтрации и объектов политики безопасности, охватывающая как классические списки доступа, так и политики межсетевых экранов нового поколения, включая отечественные решения. За основу анализа взята теоретико-множественная модель FIREMAN, расширенная в рамках работы. Помимо классических аномалий между правилами фильтрации, набор обнаруживаемых ошибок расширен аномалиями отдельного правила — выключенными, истёкшими и неиспользуемыми правилами — и ошибками уровня объектов политики безопасности, на которые ссылаются правила. Отдельно рассмотрена обработка правил с квалификаторами, не выразимыми в модели: для них применяется консервативный анализ через нижнее и верхнее замыкания, гарантирующий отсутствие ложных критических ошибок. Разработанный анализатор был реализован и интегрирован в систему инвентаризации и валидации сетевых инфраструктур, разрабатываемую в ООО «Дипси Нетворкс». Проведённое нагрузочное тестирование показало применимость анализатора к анализу больших наборов правил межсетевых экранов нового поколения.

Содержание

1	Введение	4
2	Постановка задачи	7
3	Обзор существующих решений	8
3.1	Правила фильтрации	8
3.2	Действия в правилах	12
3.3	Качество конфигураций в реальной эксплуатации	13
3.4	Типы ошибок и аномалий в наборах правил	15
3.5	Методы анализа правил фильтрации	19
3.5.1	Попарный анализ	19
3.5.2	Query-based методы	20
3.5.3	Символьные методы анализа политик	20
3.5.4	SAT-решатели	22
3.5.5	Верификация с тернарной логикой условий сопоставления	23
3.5.6	Верификация политик в SDN	24
3.5.7	Визуальный аудит наборов правил	25
3.5.8	Анализ на основе журналов трафика	25
3.6	Анализ достижимости	26
3.7	Оптимизация политик: переупорядочивание и проектирование	27
4	Исследование и построение решения задачи	29
4.1	Унифицированная модель представления правил фильтрации и объектов политики безопасности	29
4.1.1	Политика безопасности	29
4.1.2	Модель правил фильтрации	30
4.1.3	Объекты политики безопасности	43
4.2	Обнаруживаемые классы ошибок	52
4.3	Алгоритм поиска аномалий между правилами	55
5	Описание практической части	64
5.1	Интеграция в систему инвентаризации и валидации сетевых инфраструктур	64
5.2	Сбор данных с устройств и их нормализация	65
5.3	Нагрузочное тестирование	68
6	Заключение	71

1 Введение

В современном мире постоянно растёт число киберугроз и атак на информационные системы. По данным исследования глобального центра исследований и анализа угроз Kaspersky GReAT [] 2025 года, количество атак на финансовые системы выросло на 43% по сравнению с 2024 годом в России. Сетевая безопасность является одним из ключевых элементов защиты информационных систем от киберугроз.

В последние 20 лет активно развивается рынок средств защиты информационных систем. Компании развивают не только решения для конечных пользователей (end-point security), но и решения для сетевого уровня (network security). Средства защиты применяются не только на привычных сетевых устройствах: роутерах или свитчах, но и применяются специализированные отдельные сетевые устройства: межсетевые экраны (firewalls). Межсетевые экраны обеспечивают защиту сети в соответствии с политикой безопасности. Под политикой безопасности сетевого устройства понимается формализованное описание того, какой трафик и при каких условиях является допустимым с точки зрения безопасности организации. Политика может охватывать не только фильтрацию трафика, но и смежные механизмы: трансляцию сетевых адресов (NAT), SSL-инспекцию зашифрованных соединений и другие. Центральным элементом политики является упорядоченный набор правил фильтрации. Каждое правило состоит из условия — набора квалификаторов пакета или сессии, которым он должен соответствовать, — и действия (например, разрешить, запретить). Реализация поиска подходящего правила может быть реализована по разному, но поведение совпадает с моделью «first-match»: применяется первое подходящее правило. Если ни одно правило не сработало, применяется действие по умолчанию — как правило, запрет.

В текущий момент можно выделить несколько видов межсетевых экранов, но границы между классами устройств на рынке не всегда однозначны.

Классический межсетевой экран реализует политику доступа с помощью правил фильтрации сетевого трафика, имеющие квалификаторы сетевого и транспортного уровней модели OSI []: диапазоны IP адресов, номеров протоколов и набора портов (для UDP и TCP). Такие решения применяют для разграничения сегментов сети и контроля доступом. Аналогичный механизм фильтрации трафика реализован в виде списков контроля доступа (Access Control List, ACL) на маршрутизаторах и коммутаторах, где ACL определяют, какой трафик разрешён или запрещён на конкретном интерфейсе. Однако классический межсетевой экран имеет значительный недостаток: его правило нельзя непосредственно связать с конкретным прикладным сервисом или пользователю (субъекту) в полной мере.

Под **универсальным шлюзом безопасности** (Unified Threat Management) традиционно понимают узел объединяющий на одном аппаратном комплексе или в рамках одного программного решения большое количество средств защиты: межсетевая фильтрация, системы обнаружения и предотвращения вторжений (IDPS), антивирусная и URL фильтрация, организация защищённых каналов (VPN), анализ электронной

почты. Администрирование ведётся через единый интерфейс. Для малых и средних организаций такой подход может быть более экономически выгодным и удобным, чем поддержка набора функционально близких, но технически раздельных систем.

Межсетевой экран нового поколения (Next Generation Firewall, NGFW) является приемником UTM и расширяет возможности классического межсетевого экрана: идентификация приложений, сервисов, пользователей. Данный тип экрана включает в себя все современные механизмы безопасности: глубокий анализ пакетов (Deep Packet Inspection, DPI) с контролем приложений, систему обнаружения и предотвращения вторжений (IDPS), SSL-инспекцию и гибкое управление политиками безопасности.

Межсетевой экран веб-приложений (Web Application Firewall, WAF) специализированное устройство для защиты сервисов, работающих по протоколам HTTP и HTTPS: проверке подвергаются запросы и ответы с целью блокирования атак на типовые веб-уязвимости. Данное устройство не заменяет сетевой межсетевой экран, а дополняет его, так как разграничение доступа между подсетями и реализация единой периметровой политики безопасности остаются задачами классического МЭ или NGFW.

В России активно развивается рынок межсетевых экранов нового поколения, появляются решения от разных компаний: Kaspersky, Positive Technologies, Idesco и др. Появление новых отечественных NGFW во многом связано с уходом с рынка ряда зарубежных вендоров (Check Point, Fortinet, Palo Alto Networks) и дальнейшим курсом на импортозамещение, вследствие чего организации вынуждены переходить на российские решения. При этом импортозамещение затрагивает не только сегмент NGFW и других средств защиты, но и базовое сетевое оборудование, такое как маршрутизаторы и коммутаторы. Вместе с заменой устройств приходится переносить и их конфигурации, что порождает ряд трудностей.

Во-первых, модели правил у разных вендоров отличаются, и по составу квалификаторов в правилах, и способу их задания: одни оперируют набором сетевых квалификаторов (адреса, протокол, порты), другие поддерживают именованные объекты, группы адресов или наборы портов. Прямой перенос правил «один-к-одному» часто оказывается невозможным или приводит к семантически неэквивалентной политике: правила могут быть избыточными, перекрываться или, напротив, оставлять непокрытое множество трафика. Во-вторых, в процессе миграции легко допустить ошибки конфигурации, которые не проявляются сразу, но впоследствии приводят либо к нарушению работы необходимых сервисов и приложений, либо к появлению уязвимостей в инфраструктуре организации.

Отдельной проблемой является эксплуатация уже существующих решений. Многие организации продолжают использовать классические межсетевые экраны и устройства с ACL, введенные в эксплуатацию несколько лет назад или даже десятилетия назад. За время эксплуатации в таких политиках накапливается значительное число правил. Некоторые из них были добавлены «временно» и были забыты, другие заводи-

лись по заявке пользователей, которые уже не работают в организации. Также могут накапливаться и ошибки в политике безопасности: дублирующиеся правила, перекрывающиеся правила, затенения правил и т.д. Объём политик безопасности в крупных корпоративных сетях нередко исчисляется десятками тысяч правил фильтрации, что делает ручной аудит политик практически невозможным, а попытки внести изменения — очень рискованными.

Таким образом, задача автоматизированного анализа правил фильтрации — поиска неиспользуемых правил, конфликтов, избыточности и несогласованностей в политиках безопасности — остаётся актуальной не только при сопровождении эксплуатируемой инфраструктуры, но и при миграции на новые сетевые устройства. Разработка системы, способной формально верифицировать правила фильтрации, позволяет гарантировать корректность и полноту фильтрации трафика. Это позволит организациям повысить качество и надёжность сетевой защиты, минимизировать риски, возникающие при изменении политики безопасности, и снизить трудозатраты инженеров, эксплуатирующих сетевую инфраструктуру.

2 Постановка задачи

Целью данной работы является разработка системы автоматического анализа и верификации правил фильтрации сетевого трафика для выявления ошибок конфигурации.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести исследование существующих методов анализа правил фильтрации.
2. Разработать унифицированную вендору-независимую модель представления правил различных форматов. Модель должна поддерживать отечественные решения.
3. Выбрать и адаптировать методы формальной верификации для анализа правил.
4. Реализовать алгоритмы обнаружения ошибок конфигурации в наборах правил.
5. Разработать прототип системы анализа правил фильтрации и его интеграцию в систему инвентаризации и валидации сетевых инфраструктур, разрабатываемую в ООО «Дипси Нетворкс»
6. Провести тестирование на реальных конфигурациях.

3 Обзор существующих решений

Системы анализа правил фильтрации развивались вместе с сетевыми устройствами. На смену устройствам с простыми пакетными фильтрами приходили устройства, отслеживающие состояние соединений (stateful), а затем появились NGFW с инспекцией приложений и другими расширенными функциями. Менялись и сами правила фильтрации: они уже не ограничивались стандартной пятёркой полей (IP-адрес источника, IP-адрес получателя, порт источника, порт получателя, протокол), а получили более широкий набор квалификаторов; появилась семантика цепочек, изменился подход к организации списка правил. Более ранние работы, ориентированные на простые пакетные фильтры, оказались неприменимы к конфигурациям современных устройств. Последующие подходы перешли от попарного анализа правил к символьным методам и от пятёрки полей — к расширенному набору квалификаторов; тем не менее каждый такой переход рассчитывался под конкретную платформу и определённый формат правил, не пытаясь учесть всё многообразие возможных конфигураций.

Для построения собственной системы в настоящем разделе рассматриваются модели правил фильтрации, уже предложенные в литературе, обсуждается сложность реальных конфигураций, систематизируются типы ошибок и аномалий, выделяемых разными авторами, и разбираются предложенные методы анализа — от попарных и геометрических до символьных, логических, формально верифицированных и распределённых. Отдельно рассматриваются методы оптимизации политик: переупорядочивание правил и проектирование политики с нуля.

3.1 Правила фильтрации

В наиболее общем виде правило фильтрации представляет собой пару $\langle P, a \rangle$, где P — предикат над набором атрибутов сетевого пакета (адрес отправителя, адрес получателя, протокол, порты и иные поля) и контекста установленной сессии (состояние соединения, идентификаторы приложений, URL-категории и т.п.), а a — действие, применяемое к пакету/сессии в случае истинности предиката.

Правила фильтрации организованы в упорядоченные наборы (списки), которые применяются к каждому проходящему пакету последовательно: первое сработавшее правило определяет результат обработки пакета. Из-за простоты такого определения на правило фильтрации не накладываются практически никакие ограничения, и производители сетевых устройств реализуют данный функционал по-своему: различаются состав квалификаторов и способы их задания, множество возможных действий и принципы организации списков правил.

Примеры правил фильтрации

Для примера рассмотрим типовую политику фильтрации трафика на разных типах сетевых устройств. Возьмём следующую задачу: *разрешить HTTPS-трафик из*

внутренней сети 192.168.1.0/24 к веб-серверу 10.0.0.10 и заблокировать всё остальное.

В качестве классических правил фильтрации рассмотрим списки контроля доступа (ACL) на маршрутизаторах и коммутаторах **CISCO IOS**. Данная политика записывается в виде двух строк `access-list`'а, привязываемого к интерфейсу:

```
access-list 101 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.10 eq 443
access-list 101 deny ip any any
interface GigabitEthernet0/1
 ip access-group 101 in
```

Первая команда разрешает TCP-трафик из подсети 192.168.1.0/24 в подсеть 10.0.0.10/32 на порт 443. Вторая команда запрещает всё остальное. При этом данный набор правил действует только на входящий трафик интерфейса GigabitEthernet0/1.

На устройствах с Linux та же политика записывается с помощью команд `iptables`:

```
iptables -A FORWARD -s 192.168.1.0/24 -d 10.0.0.10 \
        -p tcp --dport 443 -j ACCEPT
iptables -A FORWARD -j DROP
```

Каждая команда добавляет (`-A`, *append*) одно правило в конец цепочки **FORWARD** — предопределённой цепочки таблицы **filter**, через которую проходит транзитный трафик. Первое правило задаёт условие сопоставления: адрес отправителя из подсети 192.168.1.0/24 (`-s`), адрес получателя 10.0.0.10 (`-d`), протокол TCP (`-p tcp`) и порт назначения 443 (`-dport`). При выполнении всех условий применяется действие `-j ACCEPT`, разрешающее пакет. Второе правило не содержит условий (а значит, сопоставляется любому пакету) и применяет действие **DROP** — блокировка без уведомления отправителя.

В отличие от Cisco IOS в `iptables` действие правила задаётся не отдельным ключевым словом (**permit/deny**), а так называемой *целью* (**target**), указываемой после флага `-j`. Целью может быть встроенное действие — **ACCEPT** (разрешить), **DROP** (блокировка без уведомления отправителя), **REJECT** (блокировка с ICMP-уведомлением), **LOG** (записать пакет в журнал), **RETURN** (возврат к предыдущей цепочке) — либо имя пользовательской цепочки правил; в этом случае происходит переход на указанную цепочку, а при достижении её конца обработка возвращается обратно. Помимо предопределённых цепочек **INPUT**, **OUTPUT** и **FORWARD**, через которые автоматически проходит соответствующий трафик, администратор может создавать собственные цепочки и использовать их как переиспользуемые блоки правил, что делает `iptables` существенно более выразительным средством, чем плоский ACL.

В **NGFW** правило стараются задавать над именованными объектами, зонами и идентификаторами приложений уровня L7, а не над портами. В качестве примера приведём фрагмент конфигурации FortiGate, реализующий описанную выше политику:

```

config firewall address
    edit "Internal-Net"
        set subnet 192.168.1.0 255.255.255.0
    next
    edit "Web-Server"
        set subnet 10.0.0.10 255.255.255.255
    next
end

config application list
    edit "Allow-HTTPS-only"
        config entries
            edit 1
                set application 40568
            next
        end
        set other-application-action block
    next
end

config firewall policy
    edit 1
        set name "Allow-HTTPS"
        set srcintf "internal"
        set dstintf "wan1"
        set srcaddr "Internal-Net"
        set dstaddr "Web-Server"
        set service "ALL"
        set action accept
        set application-list "Allow-HTTPS-only"
        set ssl-ssh-profile "deep-inspection"
        set utm-status enable
    next
    edit 2
        set name "Deny-Default"
        set srcintf "any"
        set dstintf "any"
        set srcaddr "all"
        set dstaddr "all"
        set service "ALL"

```

```
    set action deny
next
end
```

Конфигурация FortiGate представляет собой иерархию секций `config ... end`, внутри которых отдельные объекты задаются блоками `edit name ... next`. В секции `firewall address` создаются именованные сетевые объекты. В секции `application list` описывается профиль контроля приложений `Allow-HTTPS-only`, разрешающий лишь приложение с числовым идентификатором сигнатуры HTTPS (40568) и блокирующий любые другие приложения (`set other-application-action block`). В секции `firewall policy` задаются два правила с приоритетами 1 и 2. Каждое правило связывает пару интерфейсов (`srcintf/dstintf`) и пару адресных объектов (`srcaddr/dstaddr`). В отличие от Cisco IOS, правило не привязывается к одному интерфейсу с направлением, а явно указывает входящий и исходящий интерфейсы. У других производителей встречается привязка правил к зонам безопасности (*zone-based*). Под зоной безопасности понимается группа интерфейсов, объединённых в единую логическую сущность.

Принципиальное отличие от классических ACL в том, что классификация трафика как HTTPS выполняется не по номеру TCP-порта, а методами глубокой инспекции пакетов (DPI). Поле `service` в правиле отвечает только за фильтрацию по L4 модели OSI (протокол и порт). Идентификация приложения задаётся с помощью профиля контроля приложений (`application-list`), в котором FortiGate определяет HTTPS даже при использовании нестандартного порта. Поскольку HTTPS-трафик представляет собой связку SSL и HTTP, для определения HTTP внутри SSL-сессии необходимо использовать профиль SSL/TLS-инспекции (`ssl-ssh-profile "deep-inspection"`). Опция `utm-status enable` включает применение UTM-профилей (контроль приложений, IPS, антивирус и пр.) к данному правилу.

Уже на этих простейших примерах видны основные различия и эволюция в подходах к описанию правил фильтрации сетевого трафика. Во-первых, меняется способ привязки правила внутри политик: в Cisco IOS список правил жёстко связан с конкретным интерфейсом и направлением трафика, в iptables — с цепочкой обработки в подсистеме ядра, а в FortiGate правило формулируется уже над парой интерфейсов или зон безопасности и тем самым отделяется от физической топологии. Во-вторых, меняется форма задания сетевых квалификаторов: первые два примера опираются на примитивы — IP-адреса, маски и номера портов, — тогда как современные решения работают с именованными объектами и их группами (`Internal-Net`, `Web-Server`, `Allow-HTTPS-only`), что позволяет описывать политику в терминах организационных сущностей, а не конкретных значений.

Однако отличия могут наблюдаться и на, казалось бы, близких платформах с одинаковым видом правил: чувствительность имён объектов к регистру, использование разного вида объектов, различные нотации для масок и диапазонов, синонимы протоколов, а также различные способы задания действий и т. д. Подобные отличия должны

устраняться на этапе *нормализации* правил — приведении исходной конфигурации к единой внутренней модели правил и объектов, с которой работает система анализа правил. При нахождении новых расхождений может потребоваться не только адаптировать парсер для новой платформы, но и расширить внутреннюю модель правил, что приводит к значительным изменениям в самой системе анализа правил.

Состав квалификаторов

В простых пакетных фильтрах квалификаторы образуют пятёрку полей сетевого и транспортного уровней модели OSI: адрес и порт источника, адрес и порт получателя, протокол. Эта же пятёрка лежит в основе большинства формальных моделей анализа правил, рассматриваемых далее в настоящем разделе. Однако современные межсетевые экраны давно вышли за её пределы. Помимо классической пятёрки в правилах могут участвовать:

- названия сетевых интерфейсов с учётом направления и зон безопасности;
- состояния соединения;
- идентификаторы приложений;
- URL-категории и доменные имена;
- идентификаторы пользователей;
- временные интервалы (расписания);
- и др.

При этом значения квалификаторов могут задаваться как примитивами (например, одиночный IP-адрес, диапазон портов), так и ссылками на именованные объекты и группы. На современных устройствах настройка политики с помощью именованных объектов является хорошей практикой, позволяющей управлять политикой с привязкой к абстрактным организационным сущностям, а не конкретным IP-адресам и портам. Система анализа правил должна уметь работать как с примитивами, так и с именованными объектами и их группами.

3.2 Действия в правилах

Самая простая система действий, используемая в пакетных фильтрах, — бинарная: разрешить и запретить (*allow / deny*) прохождение трафика; именно она лежит в основе большинства моделей анализа правил [1–3]. Однако реально производители сетевых устройств предлагают существенно более широкий набор действий:

- Различные виды блокировки: *drop* — блокировка без уведомления отправителя; *reject* — блокировка с отправкой ICMP-уведомления или TCP RST;

- Действия с побочным эффектом, не определяющие судьбу пакета: *log* (запись пакета в журнал), *count* (увеличение счётчика, привязанного к правилу), *mark* (установка метки, читаемой последующими правилами и подсистемой маршрутизации), *queue* (передача пакета пользовательскому процессу для внешнего разбора);
- Управление механизмом цепочек в iptables/nftables: *jump* — переход в пользовательскую цепочку правил и *return* — возврат из неё;
- Специфичные действия для NGFW: *enforce* (требование пройти дополнительные проверки — IPS, антивирус, DLP), *tunnel* (перенаправление в зашифрованный канал) и другие;
- Комбинированные действия: *accept-and-log* / *deny-and-log*, упомянутые в работе [4].

Так, для пакета, попавшего под правило с действием *enforce*, последует проверка движками безопасности; окончательное решение (*allow* или *deny*) принимается уже не списком правил, а внешним движком безопасности, поэтому с точки зрения чисто фильтрационной модели такое правило разрешает трафик.

Для анализа конфликтов и поиска аномалий нет особого смысла различать разные действия, приводящие к блокировке, так как они одинаково конфликтуют с действиями, разрешающими трафик, и наоборот. Поэтому достаточно различать только два класса — *allow* и *deny*, а другие действия можно сводить к одному из этих двух классов, так как побочные эффекты этих действий не влияют на конфликты между правилами. Однако для некоторых сценариев имеет смысл сохранить различие между действиями при анализе множеств, которые относятся к конкретному действию.

В работе Diekmann и др. [5] было показано, что при формализации правил iptables достаточно только двух действий — *allow* и *deny*: действия с побочным эффектом (*log*, *count* и др.) не влияют на судьбу пакета, а цепочки в отсутствие петель всегда можно развернуть, сведя исходный набор к плоскому списку правил над бинарной системой действий. Это корректно с точки зрения анализа правил фильтрации, так как системе анализа не интересен побочный эффект правила.

3.3 Качество конфигураций в реальной эксплуатации

Прежде чем переходить к определению видов различных ошибок, стоит установить природу их возникновения. Можно представить, что однажды аккуратно спроектированный набор правил эксплуатируется без накопления ошибок. Однако эмпирические данные, собранные за последние два десятилетия, количественно фиксируют закономерности деградации конфигураций.

Одной из ранних работ в данной области остаётся исследование Wool [6]. В ней автор собрал 37 наборов правил межсетевых экранов Check Point FireWall-1 из крупных организаций разных секторов и оценил их по 12 типам ошибок, описанных в работе.

Главный количественный вывод: более 90 % обследованных межсетевых экранов содержали хотя бы одну грубую ошибку, причём девять из двенадцати ошибок встречались более чем в половине конфигураций. Помимо собранной статистики Wool ввёл метрику сложности конфигурации RC :

$$RC = N_{\text{rules}} + N_{\text{objects}} + \frac{N_{\text{interfaces}}(N_{\text{interfaces}} - 1)}{2}$$

где N_{rules} , N_{objects} , $N_{\text{interfaces}}$ — количество правил, объектов и интерфейсов, используемых в наборе соответственно. Он показал, что число ошибок зависит от сложности конфигурации логарифмически: $\# \text{errors} \approx \ln(RC) + 1,5$.

Через шесть лет Wool [7] повторил исследование на расширенной выборке из 84 наборов правил Check Point и Cisco PIX, разделил ошибки на 36 категорий по четырём группам (входящий, исходящий, внутренний трафик, заведомо рискованные правила) и подтвердил корреляцию числа ошибок и сложности конфигурации. В этот раз уже потребовалась вендору-независимая метрика сложности конфигурации межсетевого экрана FC :

$$FC_p = N_{\text{lines}} - 50, \quad FC_c = N_{\text{rules}} \cdot N_{\text{interfaces}} + N_{\text{objects}},$$

где N_{lines} — число строк конфигурации правил Cisco PIX, N_{rules} , $N_{\text{interfaces}}$, N_{objects} — количество правил, интерфейсов и объектов соответственно для Check Point.

Главным результатом повторного исследования стал факт — межсетевые экраны более новых версий не позволяют администраторам совершать меньшее число ошибок. Таким образом, от необходимости в аудите конфигураций невозможно избавиться переходом на новую платформу.

Аналогичные результаты фиксируются и в индустриальных источниках: *Verizon Data Breach Investigations Report* [8] систематически отмечает ошибки конфигурации сетевых средств защиты как одну из устойчивых причин утечек данных наряду с эксплуатацией уязвимостей и социальной инженерией. Таким образом, задача автоматизированного аудита правил мотивируется не только академическими метриками, но и реальными издержками эксплуатации.

В итоге получаем три важных вывода о качестве политик безопасности и правил фильтрации в частности:

1. Ошибки в наборах правил накапливаются гарантированно: их доля в обследованных конфигурациях достаточно велика, чтобы рассматривать ручную проверку как недостаточную.
2. Основным предиктором числа ошибок является структурная сложность конфигурации, а не только число правил само по себе.
3. Проблема наличия ошибок в наборах не решается сменой платформы.

3.4 Типы ошибок и аномалий в наборах правил

Понятие ошибок и аномалий занимает ключевое место в системах анализа правил. Разные авторы вводили близкие по смыслу, но не всегда совпадающие определения, поэтому на данный момент существует несколько различных классификаций. За основу взята модель, представленная в работе Al-Shaer и Hamed [1] и формализованная Yuan и др. в работе [2]. Кроме того, Al-Shaer и Hamed распространили свою таксономию за пределы правил фильтрации [9] — на политики IPsec и QoS.

Отношения между парами правил

Al-Shaer и Hamed [1] вводят формальные отношения между парами правил (R_x и R_y) с помощью полей ($R_x[i]$ и $R_y[i]$) из 5-tuple:

- **Полностью различимые (Completely disjoint, CD)** — ни одно поле R_x и R_y не совпадает.
- **Точно совпадающие (Exactly matching, EM)** — все поля совпадают; такая пара представляет вырожденный случай, при котором правило с большим индексом гарантированно либо избыточно (если действия совпадают), либо затенено (если различаются).
- **Полностью включает (Inclusively matching, IM)** — все поля одного правила являются подмножествами соответствующих полей другого правила, но при этом хотя бы одно поле не совпадает.
- **Частично соответствует (Partially matching, PM)** — хотя бы для одного поля выполнено отношение строгого подмножества, надмножества или равенства, и существует другое поле, для которого это не выполнено.
- **Коррелирующие (Correlated, C)** — по некоторым полям правило является строгим подмножеством или совпадает с другим, а по остальным полям является строгим надмножеством.

Авторы доказывают теорему о полноте этой таксономии: других отношений между парами 5-tuple правил не существует. Та же таксономия (IM/EM/PM/CD/C) воспроизводится в более поздних обзорах, в частности у Ramesh и др. [10].

Радомский [11] расширяет приведённую таксономию добавлением отношения **соседства (adjacency)**: два правила соседствуют, если существует единственное поле, по которому их можно объединить в связный интервал, а остальные поля совпадают. Это отношение не входит в пять классов Al-Shaer и принципиально необходимо для определения union-аномалии (см. ниже): без явного отношения соседства два смежных правила выглядят как пара частично совпадающих, для которой операция объединения в одно правило не всегда определена.

Классические аномалии и их формализация

На основе перечисленных отношений Al-Shaer и Hamed определяют четыре класса аномалий между правилами R_x и R_y при $x < y$ (то есть R_x предшествует R_y в списке); используется бинарная система действий:

- **Затенение (Shadowing)** — R_x полностью включает или совпадает с R_y ; R_x и R_y имеют разные действия; R_y становится недостижимым: ни один пакет не дойдёт до R_y , поскольку будет обработан R_x . Это явная ошибка администратора.
- **Избыточность (Redundancy)** — R_x полностью включает или совпадает с R_y ; R_x и R_y имеют одинаковые действия; удаление R_y не изменит политику с точки зрения множества прошедших и заблокированных пакетов. Это не является ошибкой в строгом смысле, но указывает на избыточность набора. Acharya и др. [12] выделяют два вида избыточности: внешнюю (*external redundancy*) в указанном выше смысле и внутреннюю (*internal redundancy*) — в тех случаях, когда в поле указан набор значений, который сам по себе избыточен или записан неоптимально с точки зрения объединения. Внутренняя избыточность также не нарушает семантику, но затрудняет ручной аудит.
- **Обобщение (Generalization)** — R_x является полным включением R_y с противоположным действием. В общем случае не ошибка: нередко это сознательный паттерн дизайна «разрешить общее, кроме конкретного» (например, разрешить весь HTTP, кроме ряда подозрительных хостов). Поэтому инструмент должен сообщать о таких парах как о подозрительных, но не помечать как ошибку.
- **Корреляция (Correlation)** — R_x и R_y коррелируют, т. е. имеют пересечение по некоторому множеству пакетов, но ни одно из них не является подмножеством другого; действия у них разные. Результат обработки таких пакетов зависит от порядка правил, что почти всегда является непреднамеренным, и такие правила лучше переписать.

Yuan и др. [2] в инструменте FIREMAN переформулируют те же аномалии в терминах теоретико-множественной семантики набора правил. В работе рассматриваются не только обычные линейные списки правил, но и механизм цепочек, т. е. система действий с переходами (jump, return). Анализатор не работает с цепочками напрямую: выполняется операция выделения всех возможных путей обхода связанных списков правил. Для построения путей добавляются специальные правила с действием *pass*, означающим случай, когда обход продолжается по текущему пути; *pass* — служебное действие, вводимое для линеаризации цепочек, а не часть бинарной системы действий из таксономии Al-Shaer.

Так, для каждой позиции j на пути обхода поддерживается состояние $\langle A_j, D_j, F_j \rangle$, где A_j — множество пакетов, уже принятых предшествующими правилами, D_j — множество отброшенных, F_j — множество пакетов, ушедших на альтернативный путь; мно-

жество пакетов, доступных для рассмотрения текущим правилом, определяется как $R_j = I \cap \neg(A_j \cup D_j \cup F_j)$, где I — общее множество входящих пакетов.

Тогда для правила $\langle P_j, \text{accept} \rangle$ различают следующие случаи:

- $P_j \subseteq R_j$ — правило корректное; оно определяет действие для нового множества пакетов, не пересекающегося с предшествующими правилами;
- $P_j \cap R_j = \emptyset$ — правило не сработает ни на одном пакете (ошибка). Подслучаи:
 - $P_j \subseteq D_j$ — затенение (*shadowing*): правило должно принимать пакеты, которые уже отброшены предыдущими правилами;
 - $P_j \cap D_j = \emptyset$ — избыточность (*redundancy*): все пакеты P_j уже приняты предшествующими правилами либо ушли на альтернативный путь;
 - иначе — избыточность и корреляция (*redundancy + correlation*): часть пакетов уже отброшена, остальные приняты или ушли на альтернативный путь;
- $P_j \not\subseteq R_j \wedge P_j \cap R_j \neq \emptyset$ — правило сработает не на всех заявленных пакетах. Возможны (не взаимоисключающие) подслучаи:
 - $P_j \cap D_j \neq \emptyset$ — корреляция (*correlation*, предупреждение): часть пакетов, которые правило намерено принять, уже отброшена предшествующими правилами;
 - существует более раннее правило $\langle P_x, \text{deny} \rangle$ при $P_x \subseteq P_j$ — обобщение (*generalization*, предупреждение): правило j обобщает правило x с противоположным действием;
 - $P_j \cap A_j \neq \emptyset$ и существует более раннее $\langle P_x, \text{accept} \rangle$ при $P_x \subseteq P_j$ — избыточность (*redundancy*, ошибка) правила x : его можно удалить, так как покрываемые им пакеты всё равно будут приняты текущим правилом.

Симметричная таблица справедлива для правил с действием *deny* (с заменой $A_j \leftrightarrow D_j$).

Эта формулировка эквивалентна определениям Al-Shaer для пар правил, но дополнительно охватывает случаи, когда аномалия формируется *совместным* действием нескольких правил, — именно за счёт того, что состояния A_j , D_j агрегируют эффект всех предшествующих правил, а не одного.

Помимо аномалий в рамках одного набора правил, FIREMAN формализует классы нарушений уровня сети целиком. На уровне отдельной политики выделяются *policy violation* (расхождение фактической политики с заданными белым и чёрным списками) и *inefficiency* (избыточность и неоптимальность набора правил без нарушения семантики). Для распределённых конфигураций дополнительно вводятся *inter-firewall inconsistency* (последовательные межсетевые экраны на одном маршруте принимают противоречащие решения для одного и того же пакета) и *cross-path inconsistency* (разные маршруты между одной парой узлов приводят к разным итоговым решениям из-за различающихся множеств фильтров на пути).

В настоящей работе эти классы не рассматриваются: корректный анализ достижимости в сети требует учёта маршрутизации, NAT, VPN-туннелей и динамики управляющего плана, то есть факторов вне правил фильтрации, и относится к отдельному направлению верификации сети, представленному Header Space Analysis [13], APKeep [14], Katra [15] и другими работами по анализу достижимости.

Расширения классификации

Параллельно с попарной классификацией Al-Shaer, в работах Gouda и Liu [4, 16] формулируются два структурных свойства политики, нарушения которых сами по себе образуют классы ошибок:

- **Согласованность (consistency)** — для каждого пакета определено единственное действие. Её нарушение означает, что один и тот же пакет в одних правилах разрешён, а в других — запрещён. В упорядоченных наборах правил данное нарушение сводится к классическим аномалиям *shadowing* и *correlation*.
- **Полнота (completeness)** — любой возможный пакет покрыт хотя бы одним правилом. Её нарушение означает наличие «серой зоны» — пакетов, не сопоставленных ни одному пользовательскому правилу. На реальных межсетевых экранах эта зона неявно поглощается правилом по умолчанию («запретить всё» или «разрешить всё»), поэтому неполнота не приводит к сбоям при работе, но скрывает за умолчанием намерения администратора и затрудняет аудит.

Радомский [11] дополнительно вводит ещё два вида аномалий:

- **Объединение (Union)** — два или более соседних (*adjacent*) правил с одинаковым действием, которые можно объединить в одно без изменения семантики. Возможность такого объединения опирается на отношение *adjacency*, введённое выше; без него *union*-аномалия не определима.
- **Нерелевантность (Irrelevance)** — правило, которому не сопоставляется ни один достижимый пакет (например, из-за маршрутизации или предшествующего перехвата трафика на других устройствах). По тем же причинам, что и *inter-firewall*- и *cross-path*-аномалии выше, в настоящей работе этот класс не рассматривается.

Кроме того, Радомский предлагает разделять все виды аномалий на две категории: ошибки (*errors*) — аномалии, для которых возможно автоматическое исправление без потери семантики (*shadowing*, *redundancy*), и предупреждения (*warnings*) — аномалии, требующие подтверждения администратора, поскольку могут быть осознанным паттерном дизайна (*generalization*, *correlation*, *union*).

Следует отметить, что *union*-аномалии в современной практике эксплуатации сетей представляют меньший интерес. Организация *zero-trust*-инфраструктуры [17] предполагает построение политик от объектов и привязку правил к заявкам пользователей;

объединение таких правил без потери семантики невозможно, поскольку каждое из них несёт метайнформацию (номер заявки, ответственное лицо, срок действия), которую агрегированное правило сохранить не может.

Особое место занимают классы аномалий, не выявляемые статическим анализом конфигурации и обнаружимые только при сопоставлении политики с реально наблюдаемым трафиком или журналами. Golnabi и др. [18] вводят два таких класса: *блокировка существующего сервиса* (политика блокирует трафик, который реально требуется и наблюдается в журналах) и *разрешение трафика к несуществующему сервису* (правило разрешает трафик, для которого не существует целевой системы). Помимо этого, авторы вводят понятия **затухающих правил (decaying)** — правил, доля трафика которых статистически снижается со временем, что указывает на потерю их актуальности, и **доминирующих правил (dominant)** — правил, обрабатывающих основную долю трафика и потому критически важных для производительности. К той же категории по своей природе относятся **устаревшие правила (stale)**, оставшиеся от прежних конфигураций сервисов, давно выведенных из эксплуатации; такая аномалия зафиксирована Diekmann и др. [5] в одном из реальных кейсов (правило для удалённого SSH-доступа, оставшееся в политике после окончания соответствующего эксперимента). Все эти классы требуют данных журналов или инвентаризации сервисов.

3.5 Методы анализа правил фильтрации

3.5.1 Попарный анализ

Первые работы были посвящены попарному анализу правил с целью обнаружения аномалий. Al-Shaer и Hamed [1], помимо введения упомянутой выше таксономии, реализовали инструмент Firewall Policy Advisor. Чтобы ускорить попарный анализ, авторы строят *policy tree* — дерево, в котором каждый уровень соответствует одному полю фильтра (protocol, src_ip, src_port, dst_ip, dst_port), а узлы на уровне — значениям этого поля, фактически встречающимся в правилах. Каждое правило размещается в дереве как путь от корня к листу: первое поле выбирает узел на верхнем уровне, второе — на следующем, и так далее до листа, в котором фиксируется само правило и его действие. Анализ ведётся попарно, но дерево позволяет отсекаать заведомо несвязанные пары уже на верхних уровнях. Для каждой пары правил на каждом уровне сравниваются значения соответствующего поля и определяется отношение по классификации IM/EM/PM/CD/C. Спуск прекращается, как только получено CD на каком-то уровне. По результатам классификации и действий правил определяется тип аномалии.

Принципиальное ограничение попарного подхода — неспособность обнаруживать аномалии, формируемые совместным действием трёх и более правил. Это ограничение и привело к появлению символьных методов, рассматривающих набор правил как единое целое.

3.5.2 Query-based методы

Параллельно с попарным анализом сформировалось семейство *query-based* инструментов: Fang (Mayer, Wool и Ziskind) [19], а также позднее коммерческий Lumeta. Fang парсит реальные конфигурационные файлы межсетевых экранов разных вендоров (Cisco, Lucent), строит общую внутреннюю модель топологии и политик, и позволяет администратору задавать запросы вида «разрешает ли политика сервис s от a к b ?»; учитываются маршрутизация, NAT. Эта работа впервые реализовала статический аналитический инструмент, не требующий активного зондирования сети. Однако Fang проверяет лишь конкретные запросы пользователя, а не всё пространство пакетов: пропущенные сценарии остаются без покрытия.

3.5.3 Символьные методы анализа политик

Под *символьными методами* понимаются методы, в которых вся политика представляется единым *предикатом* над полями заголовка пакета, а теоретико-множественные операции над множествами пакетов выполняются как булевы операции над формулами. На этом представлении работает *symbolic model checking* — метод формальной верификации, в котором множества состояний системы и переходы между ними не перечисляются явно, а задаются булевыми формулами (как правило, BDD), и проверка свойств сводится к булевым операциям над этими формулами. В отличие от точечных запросов, такой подход покрывает всё пространство пакетов одним вычислением и снимает основное ограничение попарных методов: аномалия любого порядка сводится к проверке булевых соотношений между предикатами, а не к перебору пар или троек правил.

Революционной для области стала работа Yuan и др. [2] *FIREMAN: A Toolkit for FIREwall Modeling and ANalysis*. Авторы применили *symbolic model checking* к набору правил межсетевого экрана: «состояние» — это уже накопленные множества принятых, отброшенных и ушедших на альтернативный путь пакетов (механизм цепочек), а «переход» — применение очередного правила. Каждое правило записывается в виде $\langle P, \text{action} \rangle$, где P — предикат над полями пакета, а действие принадлежит множеству $\{\text{accept}, \text{deny}, \text{chain } Y, \text{return}\}$. Правила обновления состояния $\langle A, D, F \rangle$, введённого выше:

$$\begin{aligned}\langle A, D, F \rangle, \langle P, \text{accept} \rangle &\vdash \langle A \cup (R \cap P), D, F \rangle, \\ \langle A, D, F \rangle, \langle P, \text{deny} \rangle &\vdash \langle A, D \cup (R \cap P), F \rangle, \\ \langle A, D, F \rangle, \langle P, \text{pass} \rangle &\vdash \langle A, D, F \cup (R \cap \neg P) \rangle.\end{aligned}$$

Так как при наличии цепочек один ACL порождает несколько путей обхода $path_i$ (см. 3.4), каждый из которых даёт свои множества A_{path_i} и D_{path_i} , итоговые множества для всего ACL получаются объединением по путям: $A_{ACL} = \bigcup_i A_{path_i}$ и $D_{ACL} = \bigcup_i D_{path_i}$; они удовлетворяют свойству $A_{ACL} \cup D_{ACL} = I_{ACL}$. Все проверки на аномалии (приведённые в 3.4) сводятся к булевым операциям над предикатами. Ключ-

чевой инженерной находкой FIREMAN стало использование бинарных решающих диаграмм (BDD) из библиотеки BuDDy для канонического представления предикатов; операции пересечения, объединения и проверки включения сводятся к стандартным операциям над BDD. Поскольку число состояний межсетевого экрана конечно, символьная верификация (model checking) гарантирует одновременно полноту и корректность.

FIREMAN рассматривает не только одиночный межсетевой экран, но и распределённую сеть. Для последовательно соединённых межсетевых экранов справедливы соотношения $A_{\text{serial}} = \bigcap A_{\text{acl}}$, $D_{\text{serial}} = \bigcup D_{\text{acl}}$, а для параллельно соединённых — $A_{\text{parallel}} = \bigcup A_{\text{acl}}$, $D_{\text{parallel}} = \bigcap D_{\text{acl}}$. Эти выражения позволяют построить дерево распространения пакетов из любой точки сети и сравнить агрегированную политику с заданными чёрными и белыми списками для обнаружения нарушений политики и межпутевых аномалий. На реальных промышленных конфигурациях (PIX1 на 249 правил, PIX2 на 36, BSD1 на 94) FIREMAN обнаружил суммарно несколько десятков ранее неизвестных нарушений; сложность алгоритма анализа списка по числу правил — $O(n)$ при использовании BDD против $O(n^2)$ у Al-Shaer.

Ограничения FIREMAN: анализ выполняется без учёта состояний сессий (stateless); в распределённой схеме предполагается, что любой топологически возможный путь может быть выбран маршрутизацией, что в реальных сетях даёт ложные срабатывания; расширение модели и нестандартные квалификаторы в данной работе не рассматриваются.

Альтернативное символьное представление политики было предложено в более ранних работах Gouda и Liu [4] (*Firewall Decision Diagram*). FDD — это не способ описать уже существующий список правил, а способ задавать политику с нуля сразу в форме дерева решений. FDD устроен по аналогии с BDD из FIREMAN, но уровень дерева соответствует не одному биту, а целому полю фильтра (адрес, порт, протокол); рёбра при этом помечены непустыми подмножествами области соответствующего поля, а не битами 0/1. Дополнительно требуется, чтобы для каждой вершины метки исходящих рёбер не пересекались (согласованность) и в объединении давали всю область поля (полнота). Эти два условия означают, что для любого пакета путь от корня к листу существует и единственен, поэтому в политике, описанной как FDD, ошибки согласованности и полноты невозможны по построению.

В работе Liu [20] на основе FDD построен формально верифицируемый алгоритм проверки заданных свойств политики. Свойство задаётся в виде property rule $\bigwedge_i F_i \in Y_i \rightarrow \text{decision}$ и проверяется обходом FDD; на реальных межсетевых экранах в 87 и 661 правил верификация занимает миллисекунды.

Главное ограничение FDD-ориентированного подхода в том, что диаграмма задаёт политику сразу в правильной форме, но не предоставляет способа проанализировать уже существующий список правил. Преобразование произвольного списка правил в эквивалентный FDD — отдельная задача, требующая значительной предобработки и не всегда выполнимая в общем случае. Поэтому FDD в настоящей работе как структура

данных не используется.

3.5.4 SAT-решатели

Параллельно с символьными методами развивались подходы, опирающиеся на SAT-решатели. Работа Zhang, Mahmoud, Malik и Narain [21] формулирует задачу верификации эквивалентности и проверки включения межсетевых экранов как задачу выполнимости булевых формул и решает её через SAT-решатель MiniSat. Эквивалентность двух политик F_1 и F_2 опровергается, если решатель находит пакет, на котором их результаты различаются; включение $F_1 \subseteq F_2$ — если находится пакет, разрешённый первой и запрещённый второй. Поиск избыточных правил выполняется по той же схеме: каждому правилу сопоставляется управляющий бит, и жадный алгоритм отключает максимальное множество правил, сохраняя поведение политики.

Авторы также предложили формализацию задачи синтеза минимального межсетевого экрана как QBF-задачи (квантифицированной булевой задачи): требуется найти набор параметров политики, при котором её результат совпадает с эталонной политикой на всех возможных пакетах. Эксперимент показал, что даже простейшие случаи ($N_{\text{input}} = 25$) выходят за пределы возможностей современных QBF-решателей. SAT-методы для верификации, напротив, успешно работают на конфигурациях до 26 000 правил со временем порядка минут.

Близкий по духу инструмент — Margrave (Nelson, Barratt, Dougherty, Fisler и Krishnamurthi) [22]. В отличие от подхода Zhang и др., политика в Margrave задаётся с помощью запросов, которые транслируются в спецификацию для инструмента Kodkod, сводящего задачу проверки выполнимости предиката над пакетом к SAT.

Все виды анализа в Margrave сводятся к одной операции: задаётся свойство, выраженное предикатом над пакетом, и решатель проверяет, существует ли пакет, на котором это свойство выполняется. Например, проверке отсутствия избыточности правила r соответствует свойство «пакет попадает под r и не попадает ни под одно предшествующее правило»; проверке доступности подсети — «пакет с заданным dst_ip имеет действие *permit*». Ответом SAT-решателя служит не булево значение «существует/не существует», а сам найденный пакет — конкретный набор значений всех полей заголовка. Этот пакет в работе называется *сценарием* (concrete witness): он играет роль контрпримера или подтверждающего примера, который сразу показывает, какой именно трафик соответствует проверяемому свойству.

Особый случай такого запроса — *анализ влияния изменений* (change-impact analysis). На вход подаются две версии одной и той же политики: P_1 — до редактирования, P_2 — после. Анализатор ищет пакет, на котором действия этих версий не совпадают: P_1 его разрешает, а P_2 запрещает, или наоборот. Если такой пакет не найден, то редактирование не изменило поведения политики (две версии семантически эквивалентны, несмотря на различия в тексте). Если найден — сценарий показывает трафик, для которого поведение изменилось, и тем самым делает последствия правки наглядными ещё

до её применения.

Алгоритмы анализа влияния изменений, занимающие центральное место в работах Liu [3] и Margrave, в данной работе не рассматриваются: они относятся к pre-change анализу — администратор получает картину последствий ещё до внесения правки. Разрабатываемая система ориентирована на post-change аудит: регулярную проверку уже сложившейся, эксплуатируемой конфигурации с целью обнаружения накопленных в ней ошибок.

Важным формальным результатом, отражённым в работе Acharya и Gouda [23], является то, что задачи верификации заданного свойства политики и обнаружения избыточности правила взаимно сводимы друг к другу с сохранением асимптотической сложности. Это означает, что эффективный движок для одной из задач автоматически решает и вторую, что упрощает выбор архитектуры анализатора.

3.5.5 Верификация с тернарной логикой условий сопоставления

К одной из значимых работ можно отнести исследование Diekmann и др. [5] *Verified iptables Firewall Analysis and Verification*. В ней впервые предлагается подход для работы с неподдерживаемыми квалификаторами.

Между сложностью реальной iptables-конфигурации и упрощённой моделью академических анализаторов существует значительный разрыв. Iptables в современных версиях поддерживает свыше 60 типов условий сопоставления и более 200 опций; пользовательские цепочки, conntrack-состояния, алгоритмы контентного поиска — ни один из публично доступных инструментов не обрабатывает их в полном объёме. Авторы экспериментально продемонстрировали, что инструмент ITVal на простом наборе правил NAS-устройства Synology выдаёт неверный результат, а другие инструменты вообще завершаются ошибкой.

Решение, предложенное Diekmann и др., состоит в том, чтобы построить формально верифицированный «препроцессор»: серию преобразований над набором правил iptables, каждое из которых сохраняет (или контролируемо приближает) семантику и сводит сложный набор к простой модели, доступной для анализа. Полученный подход был реализован в Isabelle/HOL и формально доказан.

Ключевая идея — *тернарная логика для неизвестных условий сопоставления*. Если значение какого-либо условия для пакета не может быть установлено (например, инструмент не поддерживает опцию), оно считается неопределённым; политика анализируется в двух приближениях — оптимистическом (неопределённое условие считается истинным) и пессимистическом (неопределённое условие считается ложным). Эти два приближения дают верхнюю и нижнюю оценки множеств разрешённых и запрещённых пакетов и позволяют делать корректные выводы даже для конфигураций, часть условий в которых выходит за пределы поддерживаемого инструментом подмножества.

Итогом нормализации служит простая модель межсетевого экрана: список плоских правил без пользовательских цепочек и без неизвестных условий, в которых из

всех полей сохранены только входной и выходной интерфейсы, IP-адреса, протокол и порты. Поверх этой модели Diekmann определяет аналитические процедуры: разбиение пространства IP-адресов на классы эквивалентности по политике и построение *сервисных матриц* — таблиц, показывающих, какие классы имеют доступ друг к другу и по каким сервисам.

В работе получены важные практические результаты. На реальной конфигурации межсетевого экрана инструмент выявил ошибку с действием по умолчанию (*default policy*) — решением, которое применяется к пакету, не сработавшему ни на одном правиле: список состоял только из permit-правил, а действие по умолчанию было оставлено равным *ассерт*, поэтому любой пакет, явно не разрешённый, всё равно пропускаться. На конфигурации сетевого хранилища Synology DiskStation, где более ранний инструмент ITVal выдавал бессодержательный результат (объединял всё адресное пространство в один класс эквивалентности и тем самым полностью терял структуру политики), разработанный инструмент корректно построил классы эквивалентности и сервисную матрицу, обнаружив, что устройство открывает наружу ряд внутренних сервисов.

При этом у данного подхода есть важный недостаток: тернарная аппроксимация может терять информацию. Если в правилах слишком много нераспознанных условий сопоставления, оптимистическое и пессимистическое приближения расходятся до тривиальных «разрешено всё» и «запрещено всё», и отчёт перестаёт быть содержательным.

3.5.6 Верификация политик в SDN

Стоит рассмотреть отдельно работы, посвящённые верификации программно-конфигурируемых сетей (Software-Defined Networks, SDN). В SDN правила фильтрации и маршрутизации не настраиваются вручную на каждом устройстве, а централизованно распределяются SDN-контроллером по управляемым им коммутаторам — обычно однородным и поддерживающим единый протокол (например, OpenFlow). Администратор не редактирует таблицы каждого коммутатора, а описывает высокоуровневые сетевые требования — например, «сегмент *A* не должен иметь доступа к сегменту *B*», — а контроллер транслирует их в правила, которые рассылает по сети. Задача верификации в такой постановке состоит в том, чтобы убедиться, что полученное распределённое состояние таблиц коммутаторов действительно реализует заданные высокоуровневые требования на любом возможном маршруте пакета.

Показательной работой является исследование Карус [24]. Автор формализует поведение каждого коммутатора как функцию, отображающую пару «пакет, входной порт» в множество выходных портов, и моделирует прохождение пакета по сети как рекурсивный обход графа коммутаторов с применением этих функций. Эталонные требования задаются в виде ограничений «пакеты класса *X* не должны попадать в сегмент *Y*»; проверка состоит в том, что для распределённой конфигурации не существует пакета и маршрута, нарушающих это ограничение. Описание оформляется в формализме TLC^+ и подаётся model checker'у TLC, который перебирает все достижимые состояния

сети.

В ранней работе коммутатор моделировался как полноценный автомат: его таблица коммутации входила в состояние верификатора, и TLC при обходе перебирал все возможные комбинации записей в таблицах всех коммутаторов сети, что приводило к комбинаторному взрыву пространства состояний. В работе 2023 года таблицы коммутаторов считаются фиксированной частью конфигурации и описываются *функционально* — как чистая функция «пакет, входной порт» → «выходные порты» без внутреннего изменяемого состояния. В результате в состояние верификатора входит только текущее положение пакета в сети, и TLC перебирает существенно меньшее число вариантов; время верификации на тех же тестовых сетях сокращается с часов до секунд.

3.5.7 Визуальный аудит наборов правил

Хорошим дополнением в области визуализации правил фильтрации стала работа Mansmann, Göbel и Cheswick [25]. Авторы показывают, что наборы правил масштаба нескольких сотен строк не поддаются ручному аудиту, и предлагают представить конфигурацию диаграммой *Sunburst* — вложенными кольцевыми сегментами вокруг общего центра. Каждое кольцо соответствует одному полю правила в выбранном порядке (список контроля доступа → действие → протокол → источник → назначение), сегмент — группе правил с общими значениями полей до этого уровня, а его угловая ширина — числу правил в группе либо, в отдельном режиме, суммарному числу срабатываний (hitcount), что наглядно выделяет горячие правила и неиспользуемые. Сами по себе такие представления не выявляют новых классов аномалий, а служат удобным интерфейсом отображения текущей политики и результатов анализа других анализаторов.

3.5.8 Анализ на основе журналов трафика

Принципиально иной подход реализован в исследовании Golnabi, Min, Khan и Al-Shaer [18], использующем динамические методы анализа межсетевого экрана. Авторы предлагают метод Mining firewall Log using Frequency (MLF) — частотный анализ записей журнала, выводящий «эффективные» правила, отражающие реально наблюдаемый трафик, и метод Filtering-Rule Generalization (FRG) — объединение близких правил в одно более общее за счёт замены конкретных значений на диапазоны портов, подсети или ANY. На реальных журналах авторы показали, что эффективное поведение политики (множество правил, реально срабатывающих на наблюдаемом трафике) существенно отличается от её формальной записи: значительная часть правил никогда не срабатывает, а небольшая часть обрабатывает основную долю трафика; это даёт практическое основание для очистки набора и переупорядочивания правил по частоте. Этот подход дополняет статический анализ, но не заменяет его.

3.6 Анализ достижимости

Отдельным направлением, тесно связанным с анализом политик межсетевых экранов, является *анализ достижимости* (reachability analysis) — проверка того, какие классы пакетов могут пройти от заданного источника до заданного приёмника через всю последовательность сетевых устройств с учётом таблиц маршрутизации, ACL и преобразований заголовков. Хотя сама задача формально отличается от анализа отдельных правил, используемые в ней представления и кодирование политик сильно пересекаются.

Фундаментом этого направления служит работа Kazemian, Varghese и McKeown [13] *Header Space Analysis* (NSDI 2012). Авторы предлагают формализм без априорного знания структуры заголовков: заголовок пакета рассматривается как точка в пространстве $\{0, 1\}^L$, всё пространство заголовков обозначается $P = \{0, 1\}^L$, а каждое сетевое устройство моделируется *transfer function* $T : P \rightarrow 2^P$, преобразующей подмножества заголовков. Множества заголовков представляются с помощью тернарной логики, и для них определена алгебра пересечений и дополнений, позволяющая статически проверять достижимость, обнаруживать петли в сети и нарушения изоляции необходимых сервисов. На реальной кампусной сети Стэнфорда все петли найдены за время порядка десяти минут, проверка достижимости между двумя подсетями — за 13 секунд.

Ключевая для области абстракция — *классы эквивалентности* (equivalence classes): разбиение пространства заголовков на минимальные попарно непересекающиеся подмножества, на которых все устройства сети ведут себя одинаково, благодаря чему проверка достижимости сводится к операциям над конечным числом классов, а не над всем пространством. В минимальном виде (*atomic predicates*) эта абстракция введена в AP Verifier; работа Zhang и др. [14] *APKeep* развивает её, предлагая модульную модель устройства Port-Predicate Map и алгоритмы *инкрементального* сопровождения классов: при изменении правила существующие классы лишь расщепляются вдоль его предиката, а классы с совпадающим поведением сливаются обратно, что удерживает их число минимальным. Это снимает остроту проблемы *взрыва числа классов* (EC explosion) на многополевых ACL-правилах — в экспериментах APKeep набор всего из 686 ACL-правил породил у методов на основе диапазонов до 15 миллионов классов — и позволяет обрабатывать реальные потоки обновлений с задержкой порядка миллисекунд на изменение, что делает APKeep применимым в режиме непрерывного мониторинга.

Современное продолжение — работа Beckett и Gupta [15] *Katra* (NSDI 2022), распространяющая инкрементальную верификацию на *многоуровневые* сети, в которых протоколы инкапсуляции (VXLAN, MPLS, IP-in-IP) порождают стеки вложенных заголовков. Наивное моделирование такого стека как набора N копий заголовка приводит к крайне неэффективному представлению, а число уровней N к тому же не всегда известно заранее. Katra вместо этого хранит *декомпозированное* представление — стек заголовков представляется как стек множеств заголовков — и вводит *частичные классы эквивалентности* (partial equivalence classes), вычисляемые по верхушке стека;

межуровневые зависимости при этом не разворачиваются заранее, а лениво “чинятся” по мере обхода сети. Это позволяет анализировать достижимость в сетях с туннелями за время, сопоставимое с одноуровневым случаем (по оценкам авторов, в 5–32 раза быстрее наивного подхода уже при двух уровнях).

3.7 Оптимизация политик: переупорядочивание и проектирование

Стоит также отметить направление работ, которое не связано с поиском аномалий в правилах фильтрации напрямую, но позволяет улучшить качество политик — либо за счёт перестройки порядка правил, либо за счёт построения политики с нуля по высокоуровневой спецификации.

Переупорядочивание правил

Значительная часть работ посвящена задачам оптимизации производительности межсетевого экрана через переупорядочивание правил. Работа Fulp [26] представляет зависимости между правилами в виде направленного ациклического графа: для двух правил R_i и R_j с $i < j$ ребро $R_i \rightarrow R_j$ существует тогда и только тогда, когда их области пересекаются и действия различны. Любая топологическая сортировка такого графа даёт семантически эквивалентный набор правил, что задаёт пространство допустимых перестановок и обосновывает корректность жадного алгоритма переупорядочивания по частоте срабатывания. Nemed и Al-Shaer [27] развивают эту линию и доказывают, что задача переупорядочивания правил при наличии семантических ограничений на перестановки — *NP-полна*; они предлагают двухслойную динамическую схему, где небольшое подмножество «горячих» правил отделяется от остальных и переупорядочивается по наблюдаемой частоте.

Эти работы являются классическими, однако с точки зрения производительности их практическая актуальность сегодня существенно ниже, чем на момент публикации. Причина в том, что современные межсетевые экраны — как программные, так и аппаратные NGFW — не выполняют последовательный перебор правил. Вместо линейного просмотра списка применяются специализированные структуры быстрого матчинга: многомерные деревья классификации пакетов, hash- и trie-индексы по полям заголовков. В результате стоимость поиска подходящего правила имеет асимптотику $O(1)$ либо логарифмическую по длине набора, а не линейную. Количественную оценку этого эффекта приводят Chomsiri, He, Nanda и Tan [28]: на наборе из 80 000 правил потери производительности линейного iptables составляют 47,66 %, тогда как при древовидной организации матчинга — лишь 7,43 %. Переупорядочивание правил по частоте срабатывания слабо влияет на время поиска. На первый план вместо производительности выходит *корректность* набора правил — отсутствие аномалий и соответствие декларируемой политике безопасности.

Проектирование политики с нуля и автоматизированный синтез

Проектирование политик с нуля и автоматизированный синтез политик основываются на высокоуровневых спецификациях политики. Structured Firewall Design Gouda и Liu [16] формализует три фундаментальные проблемы прямого проектирования набора правил — консистентность (*consistency*), полнота (*completeness*) и компактность (*compactness*) — и предлагает двухэтапную обработку: сначала пользователь описывает политику в виде FDD, затем алгоритм автоматически конвертирует её в эквивалентный компактный список правил.

Diverse Firewall Design Liu и Gouda [29] переносит на проектирование межсетевого экрана принцип *N-version programming*, известный из практики разработки критичного программного обеспечения: одно и то же требование к политике независимо реализуется несколькими командами, после чего полученные наборы правил формально сравниваются между собой. Расхождение хотя бы на одном пакете означает, что какая-то из команд неверно поняла исходные требования. Сравнение списков правил «в лоб» невозможно — порядок правил, разбиение диапазонов и перекрытия в разных версиях не совпадают, — поэтому авторы приводят все версии к канонической форме упорядоченной FDD и предлагают трёхэтапный алгоритм: *construction* (построение FDD по списку правил), *shaping* (приведение всех FDD к одинаковому порядку полей и согласованной структуре) и *comparison* (синхронный обход деревьев, выдающий *все* классы пакетов, на которых решения версий различаются). Метод гарантирует полноту обнаружения расхождений, но требует наличия нескольких независимо разработанных версий одной и той же политики.

Bringhenti и др. [30] рассматривают задачу настройки фильтрации в виртуализированной сети, заданной графом сервисов и допустимых потоков трафика между ними. По исходным требованиям безопасности (какие потоки должны быть разрешены, а какие запрещены) одновременно определяется, на каких рёбрах графа разместить пакетные фильтры и какие правила сгенерировать для каждого из них. Задача формулируется как поиск конфигурации, удовлетворяющей всем требованиям и при этом минимальной по числу фильтров и суммарному числу правил, и решается SMT-решателем. Полученная политика *корректна по построению*.

4 Исследование и построение решения задачи

В настоящем разделе строится вендору-независимое решение задачи анализа правил фильтрации. Сначала вводится унифицированная модель представления правил и объектов политики безопасности, не зависящая от формата конкретного производителя. А затем формулируются обнаруживаемые классы ошибок конфигурации и описываются алгоритмы их поиска.

4.1 Унифицированная модель представления правил фильтрации и объектов политики безопасности

4.1.1 Политика безопасности

Как уже отмечалось ранее, существует множество видов правил фильтрации, как классических ACL, так и современных политик безопасности NGFW. Для того, чтобы иметь возможность не адаптировать под каждый вид правил систему анализа, необходимо ввести унифицированную модель представления правил к которой будут приводиться все правила.

Дальнейшее изложение опирается преимущественно на примеры конфигураций зарубежных производителей (Cisco, FortiGate, Palo Alto Networks, Check Point и др.), однако это не сужает область применимости модели. Отечественные решения — UserGate NGFW, Kaspersky NGFW, Континент 4 и другие — по структуре правил фильтрации очень близки к зарубежным аналогам: они оперируют тем же набором понятий (наборы правил, действия, отправитель и получатель, сервисы, зоны, именованные объекты), различаясь в основном синтаксисом конфигурации, а не семантикой. Поэтому все приводимые далее построения модели в равной мере покрывают и отечественные решения.

Политика безопасности. Первое, что требуется ввести — это политику безопасности как сущность модели. Под политикой безопасности в настоящей работе понимается совокупность правил фильтрации и объектов, на которые эти правила ссылаются. Структурно политика одного устройства (`securityPolicy`) описывается следующим образом:

```
securityPolicy: Object
    securityRuleSets: Array
    securityObjects: Object
```

где `securityRuleSets` — упорядоченный список наборов правил фильтрации, а `securityObjects` — набор именованных объектов, на которые правила ссылаются по имени.

Введённое определение сознательно ограничено фильтрацией. Реальные конфигурации сетевых устройств содержат и иные механизмы — правила NAT и SSL-инспекции, политики QoS и прочие, — которые в общем случае также можно относить к политике

безопасности. В модели они не представлены, так как для задачи анализа правил фильтрации эти механизмы не представляют интереса. Их поддержка может быть добавлена при необходимости, без пересмотра остальной части модели.

4.1.2 Модель правил фильтрации

Набор правил и его привязка. Набор правил фильтрации является упорядоченным. Для отличия одного набора от другого используется его имя: в случае ACL Cisco IOS/IOS-XE это имя ACL, в случае iptables — имя цепочки, в случае политики безопасности NGFW — имя политики. Дополнительно может быть задано описание набора правил для удобства пользователя.

Существенно также, куда применяется набор правил: один и тот же ACL может быть привязан к интерфейсу с указанием направления (классический случай для транзитного пользовательского трафика), к одному из управляющих сервисов устройства (VTY, HTTP, SNMP, NTP — доступ к которым ограничивается без понятия “направления”), либо глобально через механизм Control Plane Policing, классифицирующий и ограничивающий трафик, адресованный самому устройству. На Cisco IOS/IOS-XE это выглядит так:

```
interface GigabitEthernet0/1
  ip access-group 101 in

line vty 0 4
  access-class 23 in
ip http access-class 30
snmp-server community public RO 50
ntp access-group peer 60

class-map match-any CP-CLASS
  match access-group 110
policy-map CP-POLICY
  class CP-CLASS
    police 8000 conform-action transmit exceed-action drop
control-plane
  service-policy input CP-POLICY
```

В случае CoPP ACL привязывается к точке применения `control-plane` не напрямую, а косвенно — через `class-map` и `policy-map`. На других платформах разнообразие способов привязки меньше: в iptables пользовательские цепочки интегрируются в общую систему через переходы на них, в NGFW политика безопасности привязывается к зонам безопасности целиком. Из приведённых примеров видно, что у привязки набора правил можно выделить три параметра: тип места применения (интерфейс, зона,

управляющий сервис, control-plane policy), имя места применения (название интерфейса, имя зоны и т.п.) и направление трафика (входящий, исходящий, ненаправленный). Этих трёх параметров достаточно, чтобы единообразно описать все способы прикрепления набора правил.

Тогда структурно элемент `securityRuleSets` можно представить следующим образом:

```
securityRuleSets: Array
  name: String
  description: String
  attachments: Array
    type: Enum
      "interface", "zone", "snmp", "vty", "ssh", "telnet", "http",
      "https", "ntp", "cpol", "unspecified"
    name: String
    direction: Enum
      "ingress", "egress", "unspecified"
  rules: Array
```

Поля `name` и `description` — имя и описание набора правил, поле `attachments` — список точек его применения, поле `rules` — упорядоченный список самих правил фильтрации. Значение `unspecified` в полях `type` и `direction` используется, когда соответствующая значение не удалось определить на устройстве или оно не поддерживается. В случае ненаправленного трафика значение `direction` также будет `unspecified`.

Правило фильтрации. Теперь перейдём к самому главному элементу модели — правилу фильтрации. Раньше уже было введено определение правила фильтрации как пары $\langle P, a \rangle$, где P — предикат над набором квалификаторов сетевого пакета и контекста установленной сессии, а a — действие, применяемое к пакету/сессии в случае истинности предиката.

Действие. Начнём с определения возможных действий в правилах фильтрации. В 3.2 было уже указаны несколько систем действий, используемых в анализаторах правил фильтрации. По своей сути в действительности у нас может быть только два действия — разрешить и запретить прохождение трафика, все остальные действия можно привести к этим двум с каким-то побочным эффектом.

Хорошим дополнительным примером служит Cisco NX-OS, где у действия `deny` в самой записи ACL может указываться модификатор `icmp-off`:

```
ip access-list ACL-INGRESS
  10 deny tcp any any eq 443 icmp-off
  20 permit ip any any
```

По умолчанию при срабатывании `deny` на NX-OS устройство отправляет отправителю ICMP-сообщение “destination unreachable”. Модификатор `icmp-off` это поведение подавляет: пакет всё равно отбрасывается, но никакого ICMP-уведомления не генерируется. Само решение по пакету — запретить прохождение — остаётся тем же, а различие проявляется только в побочном эффекте.

С точки зрения задачи анализа фильтрации такие модификаторы несущественны: они не меняют ни предиката правила, ни принимаемого по пакету решения, а значит, не влияют ни на отношения между правилами, ни на классы аномалий. Но для удобства пользователя их необходимо сохранять, чтобы он мог видеть исходное действие правила в отчётах.

Тогда в модели достаточно сохранять исходное действие правила и нормализованное действие правила в бинарной системе действий: `allow` и `deny`. Однако для поддержки механизма пользовательских цепочек iptables/nftables необходимо добавить ещё два действия — `chain` и `return`: `chain` обозначает переход на другую цепочку правил, `return` — возврат из неё. Это необходимо, чтобы сохранить информацию о том, что правило переходит на другую цепочку или возвращается из неё, и не линириализовать цепочки на стадии нормализации модели.

Структурно действие правила можно представить следующим образом:

```
action: Object
  type: Enum
        "allow", "deny", "chain", "return", "unspecified"
  original: String
```

Поле `type` содержит нормализованный тип действия, поле `original` содержит исходное действие правила в виде строки. В случае `type = chain`, то поле `original` содержит имя цепочки, на которую переходит правило. `unspecified` используется, когда тип действия не удалось определить на устройстве или оно не поддерживается.

Помимо действия у правила имеется набор квалификаторов, по которым оно сопоставляется с трафиком, а также служебные поля — групповой профиль безопасности, логирование, статистика и метаданные о правиле. Состав этих полей у разных производителей существенно различается, однако можно выделить несколько основных групп.

Отправитель и получатель. Большая часть условия правила приходится на описание отправителя и получателя трафика. На простых платформах они задаются одним адресом или подсетью, на NGFW — комбинацией зоны или интерфейса, ссылок на сетевые объекты, пользователей и идентификаторов конечных устройств.

В классическом расширенном ACL Cisco IOS отправитель и получатель сводятся к адресу или подсети, прописанным прямо в строке правила:

```
access-list 101 permit tcp 192.168.1.0 0.0.0.255 host 10.0.0.10 eq 443
```


Именованные объекты не специфичны для NGFW и встречается уже в классических ACL — например, у Cisco можно определить отдельно массив подсетей или хостов, а в правиле сослаться по имени:

```
object-group network INTERNAL_NET
  192.168.1.0 255.255.255.0
object-group network WEB_SERVERS
  host 10.0.0.10
  host 10.0.0.15
ip access-list extended OUT
  permit tcp object-group INTERNAL_NET object-group WEB_SERVERS eq 443
```

Семантика правила та же, но отправитель и получатель теперь — не значения, а *ссылки* на отдельно определённые объекты. Эта схема и переносится в современные NGFW, добавляя к сетевым объектам зоны, пользователей и устройства.

В FortiGate в условии правила появляются точки входа трафика (`srcintf/dstintf` — интерфейс или зона, и пользовательская группа из каталога:

```
edit 1
  set srcintf "Internal-Zone"
  set dstintf "DMZ-Zone"
  set srcaddr "Internal-Net"
  set dstaddr "Web-Servers"
  set groups "AD-WebAdmins"
```

В Palo Alto Networks дополнительно поддерживается отрицание отдельной зоны или адреса (`negate-source`, `negate-destination`) и привязка к идентификатору конечного устройства (`source-hip/destination-hip`, `source-device`):

```
rules "Allow-Web" {
  from [ Internal Guest ]; negate-source no;
  to Untrust;
  source any; destination any; source-user any;
  source-device [ Corporate-Laptop ];
}
```

Здесь составляющие отправителя разнесены по разным полям правила: `from` (зоны), `source` (сетевые объекты), `source-user` (пользователи), `source-device` (устройства). Между этими полями неявно действует логическое “и”: пакет должен совпасть со всеми указанными значениями одновременно. У FortiGate схема аналогичная.

Иначе устроен Check Point Security Gateway: у него Source и Destination правила — это *одна* группа, в которую через CLI добавляются объекты разной природы — зоны, сетевые объекты, пользовательские группы, идентификаторы устройств — общим списком:

```
mgmt_cli add access-rule layer "Network" position 1 name "Allow-Web" \
  source.1.name "DMZ" \
  source.2.name "Internal-Net" \
  source.3.name "AD-WebAdmins" \
  destination.1.name "Web-Servers" \
  service.1.name "https"
```

Внутри одной группы объекты Check Point комбинируются логическим “или”: пакет совпадает с правилом, если он пришёл из зоны DMZ *либо* из подсети Internal-Net *либо* от пользователя группы AD-WebAdmins. Если те же три объекта разнести по аналогичным полям Palo Alto (`from = DMZ`, `source = Internal-Net`, `source-user = AD-WebAdmins`), результирующее условие будет принципиально другим — пакет должен прийти из DMZ *и* из Internal-Net *и* от AD-WebAdmins одновременно. Семантически Check Point и Palo Alto в этой ситуации описывают разное множество допускаемого трафика, и для воспроизведения каждой из этих семантик в общей модели необходимо учитывать логическую операцию между элементами группы разной природы.

Помимо зарубежных платформ, аналогичные конструкции встречаются и в отечественных межсетевых экранах. В UserGate NGFW правила фильтрации задаются на собственном языке UPL (UserGate Policy Language), но семантика близка к описанным выше примерам. Kaspersky NGFW по семантике правил близок к Check Point и Palo Alto.

Таким образом, отправитель и получатель образуются комбинацией четырёх независимых составляющих: точки входа трафика (зона или интерфейс), сетевого объекта, пользователя и устройства, — и дополнительно режима их комбинирования (“и”/“или”). У получателя пользователя не бывает: у соединения только один владелец — тот, кто его инициировал, — а на принимающей стороне обычно стоит сервис, а не пользователь. В остальном структура симметрична.

В модели отправитель и получатель представляются следующим образом:

```
source: Object
  isLogicalCombine: Bool
  trafficEntryPoints: Array
    isNegated: Bool
    interfaceName: String
    zoneName: String
  networkObjectNames: Array
  userNames: Array
  deviceNames: Array
destination: Object
  isLogicalCombine: Bool
  trafficEntryPoints: Array
    isNegated: Bool
    interfaceName: String
```

```
zoneName:      String
networkObjectNames: Array
deviceNames:   Array
```

Поле `trafficEntryPoints` — список точек входа трафика, указанных непосредственно в правиле (в отличие от точек применения всего набора правил из `attachments`). Каждый элемент — либо зона (`zoneName`), либо интерфейс (`interfaceName`); флаг `isNegated` соответствует синтаксисам вида `negate-source`, “!” в iptables и т.п. и означает, что данная зона/интерфейс из условия правила исключается.

Поля `networkObjectNames`, `userNames`, `deviceNames` — списки имён сетевых объектов, пользователей и устройств соответственно. Все они ссылаются по имени на собранную коллекцию объектов (`securityObjects`); сами объекты описываются отдельно, а в правиле хранятся только идентификаторы — имена. Если объект задан в исходной конфигурации не именованной ссылкой, а *в самом правиле* (например, ACL Cisco IOS подсетью `192.168.1.0 0.0.0.255`), нормализатор всё равно создаёт для него запись в `securityObjects` с искусственным именем, а в правило подставляет ссылку; в качестве имени используется сам исходный текстовый фрагмент ("`192.168.1.0/24`" и т.п.), что позволяет анализатору нифицировать обращение с разными видами полей. Поле `deviceNames` соответствует понятиям `host-id/source-device`, поддерживаемым у части NGFW (Palo Alto Host-ID, Cisco TrustSec и ISE), и для платформ без такого понятия остаётся пустым. Пустой список любого из этих полей трактуется как “любое значение” (`any`).

Режим комбинирования — то самое различие, которое было разобрано на примере Palo Alto и Check Point, — задаётся флагом `isLogicalCombine`: значение `false` соответствует комбинированию через “и” (типовой случай Palo Alto, FortiGate, Cisco), значение `true` — через “или” (необходимо для воспроизведения семантики смешанной ячейки Source/Destination Check Point). Нормализатор выставляет флаг на основании семантики конкретной платформы, анализатор при обработке правила учитывает его при построении предиката.

Тип правила. Когда у правила в условии появляются зоны, у части платформ появляется возможность указать, какой именно трафик правило вообще способно сопоставить по отношению к выставленным зонам. Явно она присутствует у Palo Alto Networks: в политике безопасности правило обязано указывать `rule-type`, принимающий одно из трёх значений:

```
set rulebase security rules "Allow-East-West" rule-type intrazone
set rulebase security rules "Allow-Web"       rule-type universal
set rulebase security rules "Deny-North-South" rule-type interzone
```

Эти значения задают предварительное условие на соотношение зон отправителя и получателя, которое проверяется ещё до сопоставления конкретных зон отправителя и получателя:

- **intrazone** — правило применяется только к трафику, у которого зона отправителя и зона получателя совпадают (например, обмен между сервисами внутри одной демилитаризованной зоны DMZ);
- **interzone** — наоборот, только к трафику между *разными* зонами (типичный пример — передача из Trust в Untrust);
- **universal** — к любому трафику независимо от соотношения зон; правило применяется и к внутризонному, и к межзонному обмену.

Тем самым **rule-type** выступает дополнительным фильтром по соотношению зон, действующим поверх остальных условий правила. У других платформ отдельного поля такого рода нет, и поведение правила соответствует значению **universal**.

В модели тип правила представляется отдельным enum:

```
rule-type: Enum
    "universal", "intrazone", "interzone", "unspecified"
```

Значения **universal**, **intrazone** и **interzone** напрямую соответствуют **rule-type** Palo Alto. Значение **unspecified** при ошибках нормализации. В случае отсутствия такой настройки на платформе выставляется тип **universal** по умолчанию.

Сервисы и приложения. Помимо адресов, зон и пользователей, правило задаёт и сам трафик, к которому оно применяется. Для этого в модели используются три типа квалификаторов: сервис, приложение и категория URL.

Сервис — это совокупность параметров, идентифицирующих трафик на уровне сетевого/транспортного протокола: указание самого протокола (TCP, UDP, ICMP, GRE, ESP и т.п.) и — при наличии у этого протокола соответствующих полей — дополнительных параметров (портов отправителя и получателя для TCP/UDP, типа и кода для ICMP и т.п.). *Приложение* — это прикладной протокол уровня L7 (например, **ssl**, **web-browsing**, **ssh**), распознаваемый устройством средствами DPI и не привязанный к конкретным портам. *Категория URL* относит запрашиваемый ресурс к одной из тематических групп (новости, социальные сети, бизнес и т.п.) и применима к HTTP- и HTTPS-трафику.

Сервис поддерживается всеми платформами; распознавание приложений и категоризация URL появились вместе с NGFW и опираются на L7-инспекцию. Способов задать сервис довольно много: даже в рамках одной Cisco IOS он может быть указан в правиле, как протокол и операторы сравнения с номерами портов, как протокол и группа портов через **object-group port**, либо как именованный сервисный объект через **object-group service**:

```
ip access-list extended ACL-MIXED
  permit tcp any host 10.0.0.10 eq 443
  permit tcp any host 10.0.0.10 range 8000 8100
```

```
permit tcp any host 10.0.0.10 portgroup WEB-PORTS
permit object-group SVC-DB any host 10.0.0.20
```

```
object-group port WEB-PORTS
eq 80
eq 443
range 8000 8100
```

```
object-group service SVC-DB
tcp eq 1521
tcp eq 3306
udp eq 1434
```

В первой записи сервис задан непосредственно в правиле: протокол `tcp` и условие на порт назначения `eq 443`; во второй — то же, но с условием на диапазон портов через `range`; в третьей условие на порт вынесено в группу портов `WEB-PORTS`, а протокол по-прежнему указан в самом правиле; в четвёртой и протокол, и условия на порты упакованы в сервисный объект `SVC-DB`, охватывающий сразу несколько сервисов СУБД. На других платформах набор способов отличается, но сводится к тем же двум вариантам: сервис указан в самом правиле или вынесен в отдельный объект, на который правило ссылается по имени.

В Palo Alto Networks в одном правиле могут одновременно присутствовать ссылка на приложение L7, на сервис и на категорию URL:

```
rules "Allow-Web" {
  application [ ssl web-browsing ];
  service     application-default;
  category    [ business-and-economy news ];
  action      allow;
}
```

Здесь `application` — идентификатор протокола приложений, определяемый автоматически (порт уже не нужен), `service` — ссылка на сервисный объект, `category` — ссылка на категорию URL. У разных производителей соответствующие квалификаторы именуются по-разному (Application-ID и URL Filtering у Palo Alto, App Control и Web Filter у FortiGate, Content-ID у Check Point), но по структуре сводятся к одному и тому же: ссылка на встроенный или пользовательский объект.

В модели правила эти три квалификатора представлены как списки имён:

```
applicationNames: Array
serviceNames:     Array
urlCategoryNames: Array
```

Каждый список содержит имена объектов из соответствующих коллекций (`securityObjects`): приложений, сервисов и URL-категорий. Пустой список трактуется как “любое значение”. Для платформ, не поддерживающих L7-инспекцию, `applicationNames` и `urlCategoryNames` остаются пустыми.

Дополнительные опции. Помимо рассмотренных квалификаторов, в правилах могут встречаться и более редкие условия на отдельные поля заголовков и состояние соединения — ограничение по TCP-флагам, TTL и состоянию сессии.

Первый класс таких условий — ограничение по TCP-флагам. В `iptables` они задаются ключом `--tcp-flags`, в котором сначала перечисляется маска проверяемых флагов, затем — те из них, которые должны быть установлены:

```
iptables -A FORWARD -p tcp --tcp-flags SYN,ACK,FIN,RST SYN -j DROP
```

Это правило срабатывает на TCP-пакетах, у которых из четырёх флагов `SYN`, `ACK`, `FIN`, `RST` установлен только `SYN`, а остальные сброшены, — то есть на типичные пакеты установления соединения. На Cisco IOS аналогичный квалификатор записывается через `match-all/match-any` с перечислением установленных и снятых флагов:

```
access-list 101 permit tcp any any match-all +syn -ack -fin
```

Из обеих записей видно, что условие разбивается на три части: режим сопоставления (все указанные флаги или хотя бы один), список флагов, которые должны быть установлены, и список флагов, которые должны быть сброшены. В модели они представляются объектом `tcpFlagsOption`:

```
tcpFlagsOption: Object
  modifier: Enum
    "match_all", "match_any", "unspecified"
  flags: Array
    "ack", "psh", "rst", "fin", "syn", "urg", "unspecified"
  negatedFlags: Array
    "ack", "psh", "rst", "fin", "syn", "urg", "unspecified"
```

Поле `flags` — множество флагов, которые должны быть установлены в TCP-заголовке, `negatedFlags` — те, которые должны быть сброшены, `modifier` управляет тем, требуется ли совпадение всех указанных флагов сразу или хотя бы одного из них.

Второй класс модификаторов — ограничения по TTL. На Cisco IOS они задаются через `ttl` с операторами `eq`, `lt`, `gt`, `range`, на `iptables` — через модуль `ttl`:

! Cisco IOS

```
access-list 101 permit ip any any ttl range 30 64
```

iptables

```
iptables -A INPUT -m ttl --ttl-eq 1 -j DROP
```

Эти условия удобнее всего хранить как набор интервалов:

```
ttranges: Array
  min: Number
  max: Number
```

Каждый элемент задаёт интервал значений TTL, при попадании в который правило срабатывает; одиночные значения и односторонние ограничения представляются как интервалы с совпадающими или граничными концами.

Третий класс модификаторов — состояние соединения. В iptables оно вводится модулем `conntrack` с ключом `--ctstate`, а метка соединения проверяется модулем `connmark` с ключом `--mark`; в MikroTik RouterOS тем же целям служат ключи `connection-state` и `connection-mark`:

```
# iptables
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m connmark --mark 0x64 -j ACCEPT

# MikroTik RouterOS
/ip firewall filter add chain=forward connection-state=established,related \
                        connection-mark=voip-traffic action=accept
```

Состояние соединения — это значение из фиксированного набора, определяемое механизмом отслеживания сессий устройства; метка соединения — произвольное пользовательское значение, проставленное соединению отдельным правилом (в iptables — целью `CONNMARK` в таблице `mangle`, в RouterOS — действием `mark-connection`). Метки позволяют один раз классифицировать соединение по сложному условию (например, отнести его к VoIP-трафику или к определённом клиенту) и затем в остальных правилах ссылаться на эту классификацию по короткой метке, не повторяя исходное условие. Правило фильтрации лишь *читает* метку, выставленную трафику ранее, на предшествующих этапах обработки. В модели они представляются отдельными полями `options`:

```
connectionState: Array
  "new", "established", "related", "invalid", "untracked", "unspecified"
connectionMark: String
```

`connectionState` — список допустимых состояний соединения (пустой список трактуется как “любое состояние”), `connectionMark` — ссылка на пользовательскую метку соединения, проставленную трафику ранее.

Расписание. У части правил появляется временной квалификатор — расписание, ограничивающее срабатывание правила определёнными часами или днями недели. В

Cisco IOS оно задаётся отдельным объектом `time-range`, на который правило ссылается по имени; в FortiGate — `set schedule`:

```
! Cisco IOS
time-range BUSINESS-HOURS
  periodic weekdays 8:00 to 17:00
!
ip access-list extended ACL-SSH
  permit tcp any any eq 22 time-range BUSINESS-HOURS

# FortiGate
config firewall schedule recurring
  edit "BUSINESS-HOURS"
    set day monday tuesday wednesday thursday friday
    set start 08:00
    set end 17:00
  next
end
config firewall policy
  edit 1
    set schedule "BUSINESS-HOURS"
  next
end
```

В модели правило содержит лишь ссылку на расписание по имени; само расписание описывается отдельным объектом в `securityObjects`:

```
scheduleId: String
```

Пустое значение трактуется как “правило активно постоянно”.

Групповой профиль безопасности. После того как правило разрешило трафик, на NGFW к нему обычно применяется набор дополнительных проверок — IDPS, антивирус, фильтрация веб-контента, защита от утечек и т.п. У FortiGate, Palo Alto, Check Point эти проверки объединяются в именованный групповой профиль безопасности (`security profile`, `group profile`, `utm-profile`), на который правило ссылается:

```
# Palo Alto
rules "Allow-Web" { profile-setting { group "Default-Profile-Group"; } }

# FortiGate
config firewall policy
  edit 1
```



```

set utm-status enable
set profile-type group
set profile-group "default"
next
end

```

Настройки группового профиля безопасности не влияют на сопоставление трафику правилу, но влияют на его постобработку, и данная информация не используется статическим анализатором. В модели это представлено единственным полем — ссылкой на объект:

```
groupProfileName: String
```

Поле содержит имя группового профиля безопасности; для платформ без понятия такого профиля поле остаётся пустым.

Полнота нормализации. Состав квалификаторов у разных производителей в общем случае шире, чем поддерживает введённая модель: на устройстве могут встречаться экзотические опции сопоставления. Просто отбросить такие квалификаторы нельзя: правило с неучтённым условием в анализе нельзя считать эквивалентным правилу без этого условия — иначе можно сообщить пользователю о shadowing там, где в реальности правила покрывают разный трафик. Поэтому в модели сохраняется явная отметка о том, что часть правила не была распознана при нормализации:

```

isComplete:      Bool
unknownMatches:  Array

```

Поле `isComplete` равно `true`, если все квалификаторы исходного правила удалось отобразить в модель без потерь; в противном случае оно сбрасывается в `false`, а исходные фрагменты, которые не удалось интерпретировать, сохраняются в `unknownMatches` в виде строк — для воспроизведения в отчёте и для последующего расширения нормализатора.

Статистика и временные отметки. У правила, помимо его конфигурации, существует и динамическая информация — сколько раз правило сработало и когда, а также когда оно было создано и последний раз изменялось. У Cisco IOS команда `show access-list` возвращает только счётчик попаданий по записи ACL, без отметки времени; у FortiGate счётчики доступны через `diagnose firewall iprope show`, а у Palo Alto интерфейс “Rule Usage” даёт и счётчик, и временные отметки первого и последнего срабатывания правила. В модели эта информация собрана в отдельный блок `ruleUsage` и в две временные отметки на уровне правила:

```

ruleUsage: Object
hitCount:      Number

```

```
    lastHitTimestamp: String
    firstHitTimestamp: String
    modifiedTimestamp: String
    createdTimestamp: String
```

`hitCount` — число попаданий с момента последнего сброса счётчиков, `lastHitTimestamp` и `firstHitTimestamp` — временные отметки первого и последнего срабатывания, `modifiedTimestamp` и `createdTimestamp` — моменты создания правила и его последнего изменения. Эти поля заполняются опционально, по мере доступности данных на устройстве, и используются анализатором для классификации аномалий.

Метаинформация. Помимо предиката и действия, в правиле есть служебные поля, которые сами по себе трафик не сопоставляют, но без которых правило нельзя ни корректно обработать и отобразить в отчёте. Эту группу далее будем называть метаинформацией правила: в неё входят идентификатор, приоритет, признак включённости, текстовое описание и параметры логирования.

От приоритета и признака включённости зависит, какое правило в итоге срабатывает: приоритет определяет позицию правила в очереди обработки, а выключенное правило исключается из набора. Идентификатор служит лишь для того, чтобы однозначно отличить правило внутри набора и сослаться на него в отчётах анализатора (на Cisco IOS/IOS-XE им служит порядковый номер записи ACL, на Palo Alto и FortiGate — автоматически назначаемый идентификатор правила). Признак включённости задаётся через `set status disable` (FortiGate) и `disabled yes` (Palo Alto), описание — через `remark` (Cisco IOS) и поле `description` (NGFW).

Логирование на разных платформах включается в нескольких режимах: запись факта начала и/или конца сессии либо запись каждого попадания пакета в правило. У Palo Alto Networks за это отвечают флаги `log-start` и `log-end`, у FortiGate — параметры `set logtraffic` и `set logtraffic-start`, у Cisco IOS — модификатор `log`, дописываемый к правилу:

```
# Palo Alto
rules "Allow-Web" { log-start no; log-end yes; }
```

```
# FortiGate
config firewall policy
edit 1
    set logtraffic all
    set logtraffic-start enable
next
end
```

```
! Cisco IOS
```

```
access-list 101 deny ip 10.0.0.0 0.255.255.255 any log
```

В модели эти реквизиты находятся на верхнем уровне правила и заполняются ещё на стадии нормализации:

```
id:           String
priority:     Number
isEnabled:    Bool
description:  String
logging:      Object
    loggingAtStartSession: Bool
    loggingAtEndSession:   Bool
    loggingAtHit:          Bool
```

`id` однозначно идентифицирует правило в пределах набора, `priority` задаёт его положение в очереди обработки, `isEnabled` отражает, активно ли правило в исходной конфигурации, `description` сохраняется для удобства пользователя и на анализ не влияет. Поля объекта `logging` соответствуют трём режимам журналирования: запись в журнал при создании сессии, при её завершении и при каждом срабатывании правила. Для платформ, поддерживающих только бинарный модификатор `log`, в нормализованной модели выставляется `loggingAtHit = true`.

4.1.3 Объекты политики безопасности

Для удобства и унификации работы с данными, в правилах фильтрации не хранятся непосредственно значения квалификаторов, а ссылаются на объекты, определённые в коллекции объектов политики безопасности, даже если эти объекты не являлись объектами в исходной конфигурации. В `securityObjects` собраны девять видов таких объектов: сетевые объекты, пользователи, устройства, зоны безопасности, группы транспортных портов, сервисные объекты, приложения, URL-категории и расписания. На структурном уровне это выглядит так:

```
securityObjects: Object
    networkObjects: Array
    users:          Array
    devices:        Array
    securityZones:  Array
    transportPortsGroups: Array
    serviceObjects: Array
    applications:   Array
    urlCategories:  Array
    schedules:      Array
```

Помимо девяти перечисленных видов, как уже упоминалось ранее, могут встречаться групповые профили безопасности. В рамках данной работы их нормализация

не рассматривается, так как они не влияют на работу правила фильтрации и не используются в статическом анализе. Данного набор объектов достаточно чтобы покрыть основные виды объектов встречающихся в конфигурациях NGFW и других сетевых устройств. Рассмотрим каждый из видов объектов по отдельности.

Сетевые объекты. Сетевой объект — именованное множество IP-адресов, на которое правила ссылаются как на отправителя или получателя трафика. На разных платформах множество адресов задаётся по-разному, и прежде чем вводить его представление в модели, разберём, какими способами это вообще делается:

```
# Cisco ASA
object network INTERNAL_NET
  subnet 192.168.1.0 255.255.255.0
object network WEB_FARM_RANGE
  range 10.0.0.10 10.0.0.20

# FortiGate
config firewall address
  edit "CN-Block"
    set type geography
    set country "CN"
  next
  edit "Bank-FQDN"
    set type fqdn
    set fqdn "online.bank.example.com"
  next
end
```

Из примеров видно, что множество адресов задаётся по-разному: одиночным хостом, подсетью (CIDR или сетевая/wildcard-маска) либо произвольным интервалом “начало–конец”. Маска задаёт только выровненные блоки, поэтому для произвольных диапазонов нужна пара адресов, но wildcard-маска позволяет описать то, что не позволяет описать интервал. Отдельно стоят два способа, не задающие адреса напрямую: указание географического региона по его коду (GeoIp) и доменное имя, разрешаемое в адреса динамически через DNS. Наконец, любой из перечисленных способов может применяться с отрицанием (“все, кроме данного диапазона”), а сам сетевой объект — ссылаться на другие сетевые объекты, образуя группу, семантически равную объединению адресов вложенных объектов (в iptables эту роль играет ipset, в Palo Alto — `address-group`, в Cisco ASA и NGFW — сетевой объект, перечисляющий другие).

Модель систематизирует перечисленное. Сетевой объект представляется списком адресных компонентов `addressObjects`, итоговое множество адресов которого есть

объединение компонентов; каждый компонент задаётся ровно одним из четырёх взаимоисключающих вариантов — `addressWildcardPair`, `addressRange`, `geoIp` или `fqdn` — и может быть отрицанием. Структурно это выглядит так:

```
networkObjects: Array
  name: String
  description: String
  isExplicit: Bool
  interfaceName: String
  addressObjects: Array
    isNegated: Bool
    addressWildcardPair: Object
      ipv4: Object
        address: Number
        wildcard: Number
      ipv6: Object
        address: Array
        wildcard: Array
    addressRange: Object
      ipv4: Object
        startAddress: Number
        endAddress: Number
      ipv6: Object
        startAddress: Array
        endAddress: Array
    geoIp: Object
      countryCode: String
    fqdn: Object
      vrf: String
      useSystemDns: Bool
      dnsServerIpv4: Number
      dnsServerIpv6: Array
      domains: Array
  networkObjectNames: Array
```

Флаг `isExplicit` различает два способа появления объекта в модели. Если объект заведён пользователем в таблице именованных объектов исходной конфигурации, флаг истинен. Если же адреса указаны прямо в правиле, а соответствующий объект создан нормализатором с искусственным именем (см. выше), флаг ложен: например, `INTERNAL_NETWORK` — явный объект, а `"192.168.1.0/24"` — нет. Это различие необходимо для того, чтобы отдельно выделять исходные объекты из конфигурации для анализа, и не учитывать те объекты, которые создал нормализатор.

Флаг `isNegated` отмечает компонент как отрицаемый — такой компонент из

множества адресов не добавляется, а исключается. В вариантах `addressWildcardPair` и `addressRange` адрес и маска хранятся раздельно для IPv4 и IPv6. `geoIp` хранит лишь код региона `countryCode`, а `fqdn` — список доменных имён `domains` и параметры их разрешения: флаг `useSystemDns` (системные DNS-серверы устройства либо явно заданные `dnsServerIpv4/dnsServerIpv6`) и имя VRF `vrf`, в контексте которого выполняется разрешение. Поле `networkObjectNames` — ссылки на другие сетевые объекты, делающие данный объект группой; окончательное множество адресов — объединение собственных `addressObjects` и адресов всех включённых объектов. Поле `interfaceName` опционально и используется на платформах, где сетевой объект осмыслен лишь в контексте конкретного интерфейса, иначе оно пусто.

Отдельно следует оговорить, что варианты `geoIp` и `fqdn` задают адреса не напрямую, а через внешние данные — базу геолокации и результаты разрешения доменных имён. Эти данные не статичны и собираются с устройства дополнительно. В приведённой схеме модели для краткости не описаны: `geoIp` и `fqdn` хранят лишь исходные параметры компонента, а сопоставление их с фактическими адресами анализатор выполняет по отдельно хранимым данным на момент анализа.

Пользователи. Пользователь — именованный субъект, на которого правило ссылается как на отправителя трафика; на NGFW это позволяет строить политику не по адресам, а по учётным записям, сопоставляемым с трафиком механизмом идентификации (Palo Alto User-ID, FortiGate FSSO). Объект может перечислять учётные записи непосредственно либо ссылаться на другие объекты-пользователи, образуя группу:

```
# Palo Alto
rules "Allow-Eng" { source-user [ "corp\\alice" "corp-engineers" ]; }

# FortiGate
config firewall policy
  edit 1
    set groups "Engineering"
  next
end
```

По аналогии с сетевым объектом пользователь представляется собственным списком учётных записей и списком ссылок на другие объекты-пользователи:

```
users: Array
  name:      String
  description: String
  accounts:  Array
  userNames: Array
```

Поле **accounts** — имена учётных записей, входящих в объект непосредственно; **userNames** — ссылки на другие объекты-пользователи. Окончательный состав вычисляется как объединение собственных **accounts** и учётных записей всех включённых объектов; если **userNames** непуст, объект играет роль группы.

Устройства. Устройство — именованный идентификатор конечного оборудования, на который правило ссылается как на дополнительный признак отправителя или получателя трафика. Механизм позволяет строить политику не только по адресам и пользователям, но и по типу или принадлежности подключённого устройства: Host-ID у Palo Alto Networks, идентификаторы устройств у FortiGate, метки TrustSec/ISE у Cisco. Объект может перечислять идентификаторы устройств непосредственно либо ссылаться на другие объекты-устройства, образуя группу:

```
# Palo Alto
rules "Allow-Eng" { source-device [ "Corporate-Laptop" "Managed-Phones" ]; }

# FortiGate
config user device-group
edit "Corp-Devices"
set member "Corporate-Laptop" "Managed-Phones"
next
end
```

Содержимое самого объекта устройства у производителя для задачи анализа фильтрации не важно: анализатор оперирует устройством как именем, а не его содержимым. В модели объект представляется так:

```
devices: Array
  name:      String
  description: String
  members:   Array
  deviceNames: Array
```

Поле **members** — имена идентификаторов устройств, входящих в объект непосредственно; **deviceNames** — ссылки на другие объекты-устройства. Окончательный состав — объединение собственных **members** и устройств всех включённых объектов.

Зоны безопасности. Зона безопасности — это именованная группа интерфейсов устройства, на которую правила и наборы правил ссылаются вместо перечисления самих интерфейсов. Концепция характерна прежде всего для NGFW, но встречается и в маршрутизаторах с zone-based firewall:

```
# FortiGate
```

```

config system zone
  edit "Internal-Zone"
    set interface "port1" "port2"
  next
end

# Juniper SRX
set zone Internal
  interfaces [ ge-0/0/1.0 ge-0/0/2.0 ]

```

В модели зона представляется самостоятельным объектом, на который ссылаются по имени `zoneName` как в `attachments` набора правил, так и в `trafficEntryPoints` отправителя и получателя:

```

securityZones: Array
  name: String
  description: String
  type: Enum
    "L2", "L3", "unspecified"
  interfaces: Array

```

Поле `type` различает L2- и L3-зоны (на NGFW классов корпорации Fortinet/Cisco/Juniper это деление существенно: L2-зоны объединяют интерфейсы, работающие в режиме коммутации, L3 — в режиме маршрутизации); если разделения нет или его не удаётся восстановить, используется значение `unspecified`. Поле `interfaces` — список имён интерфейсов, входящих в зону.

Группы транспортных портов. В классических ACL и iptables группа портов задаётся прямо в правиле — одиночным портом, перечислением, диапазоном или оператором сравнения. В именованной форме она появляется на некоторых платформах, где правила ссылаются на неё по имени:

```

# Cisco ASA
object-group service WEB-PORTS tcp
  port-object eq 80
  port-object eq 443
  port-object range 8080 8090

```

Семантически группа портов — это просто множество допустимых номеров портов, которое удобнее всего представить как список интервалов:

```

transportPortsGroups: Array
  name: String
  description: String

```



```

isExplicit: Bool
ranges: Array
  min: Number
  max: Number

```

Одиночный порт записывается как интервал с совпадающими концами, операторы вида `gt/lt` — как односторонние интервалы. На группу портов правило не ссылается напрямую: она используется только внутри сервисного объекта как значение портов отправителя или получателя. Если порты указаны прямо в правиле, нормализатор заводит для них группу портов и оборачивающий её сервис. Поэтому группа портов бывает как `explicit` (заведена пользователем), так и `implicit` (порождена из правила) — это и отмечает флаг `isExplicit`.

Сервисные объекты. Сервис задаёт сочетание IP-протокола и — для UDP, TCP и ICMP протоколов — дополнительных полей. На большинстве платформ сервис именованный, но содержит одно сочетание (один транспорт), но бывает иногда допускаются составные сервисы, объединяющие несколько транспортов под одним именем:

```

# Cisco ASA
object-group service DNS
  service-object udp destination eq 53
  service-object tcp destination eq 53

```

В модели сервис представляется списком “транспортов”, каждый из которых описывает свой IP-протокол:

```

serviceObjects: Array
  name: String
  description: String
  isExplicit: Bool
  transports: Array
    ipProtocol: Number
    srcPorts: String
    dstPorts: String
    icmpType: Number
    icmpCode: Number
  serviceObjectNames: Array

```

Поле `ipProtocol` — номер IP-протокола; `srcPorts` и `dstPorts` — имена групп портов из `transportPortsGroups` (для TCP/UDP); `icmpType` и `icmpCode` — тип и код ICMP-сообщения (для ICMP). Поле `serviceObjectNames` — ссылки на другие сервисные объекты, превращающие данный в группу; окончательное множество допустимых сочетаний протокол–порты вычисляется как объединение собственных `transports` и

транспортов всех включённых сервисов. Любое из полей транспорта может быть пустым, что трактуется как “любое значение” (`any`): незадаанный `ipProtocol` делает сервис применимым ко всем IP-протоколам, а заданный лишь `icmpType` без `icmpCode` — ко всем ICMP-сообщениям указанного типа с любым кодом. Флаг `isExplicit` аналогиче другим объектам.

Приложения. Приложение — именованный прикладной протокол уровня L7, распознаваемый устройством средствами DPI независимо от используемых портов. Производители поставляют обширные встроенные базы приложений (Palo Alto Application-ID, FortiGate Application Control) и допускают пользовательские определения; объект может перечислять приложения напрямую либо ссылаться на другие объекты-приложения:

```
# Palo Alto
rules "Allow-Web" { application [ ssl web-browsing ]; }

# FortiGate
config application group
  edit "Web-Apps"
    set application 15832 16177
  next
end
```

Содержимое самого приложения (сигнатуры, зависимости, категория в базе производителя) для задачи анализа фильтрации непрозрачно: анализатор оперирует приложением как атомарным именем. В модели объект представляется так:

```
applications: Array
  name: String
  description: String
  members: Array
  applicationNames: Array
```

Поле `members` — имена приложений из базы производителя, входящих в объект непосредственно; `applicationNames` — ссылки на другие объекты-приложения. Окончательный состав — объединение собственных `members` и приложений всех включённых объектов.

URL-категории. URL-категория — именованная тематическая группа веб-ресурсов (новости, социальные сети, бизнес и т.п.), применяемая к HTTP/HTTPS-трафику. Категории берутся из базы производителя или задаются пользователем перечислением шаблонов URL; объект может перечислять категории напрямую либо ссылаться на другие объекты URL-категорий:

```
# Palo Alto
rules "Allow-Web" { category [ business-and-economy news ]; }

# FortiGate
config webfilter urlfilter
edit 1
set name "Blocked-Sites"
next
end
```

Как и приложение, URL-категория для анализатора непрозрачна и используется как атомарное имя. В модели объект представляется так:

```
urlCategories: Array
  name: String
  description: String
  members: Array
  urlCategoryNames: Array
```

Поле **members** — имена категорий, входящих в объект непосредственно; **urlCategoryNames** — ссылки на другие объекты URL-категорий. Окончательный состав — объединение собственных **members** и категорий всех включённых объектов.

Расписания. Расписание описывает временной интервал, в течение которого правило считается активным. На разных платформах расписание может быть периодическим (повторяющимся по дням недели и часам), одноразовым (с явными датами начала и конца) или описываться группой из нескольких таких расписаний:

```
# Cisco IOS
time-range BUSINESS-HOURS
periodic weekdays 8:00 to 17:00
periodic saturday 9:00 to 13:00

# FortiGate
config firewall schedule onetime
edit "Maintenance-Window"
set start 2026/05/15 22:00
set end 2026/05/16 04:00
next
end
```

В модели три формы расписания объединены под одной структурой, но различаются через поле **type**:

```

schedules: Array
  id: String
  name: String
  isExplicit: Bool
  description: String
  type: Enum
    "recurring", "oneTime", "group", "unspecified"
  recurring: Object
    timeRanges: Array
      dayOfWeek: String
      startTime: String
      endTime: String
  oneTime: Object
    startDate: String
    endDate: String
    expirationNotificationStart: Number
  group: Object
    recurringScheduleIds: Array
    oneTimeScheduleIds: Array

```

Заполнен ровно один из трёх вариантов в соответствии со значением `type`: `recurring` — список интервалов день недели + начало/конец, `oneTime` — одноразовый интервал с датами и заблаговременным уведомлением об истечении, `group` — объединение ссылок на уже определённые расписания других двух типов. Этого набора достаточно, чтобы вместить временные конструкции основных платформ, не вводя для каждой свою специфическую форму. Флаг `isExplicit` аналогичен другим объектам.

4.2 Обнаруживаемые классы ошибок

После того как политика безопасности устройства приведена к унифицированной модели, над ней можно выполнять анализ. Все обнаруживаемые отклонения делятся на две группы: уровня правил и уровня объектов. В данной работе используется схема разделения аномалий по уровню критичности: ошибочные, предупреждающие и информационные. Ошибочные аномалии — это отклонения, которые однозначно можно интерпретировать как нарушение семантики политики безопасности. Предупреждающие аномалии — это отклонения, которые могут привести к проблемам в работе сети, но не являются критическими или не требуют исправления из-за дизайна политики безопасности. Информационные аномалии — это отклонения, которые сами по себе не являются нарушениями политики безопасности, но могут быть полезны для аудита конфигураций.

Ошибки уровня правил. Анализ выполняется отдельно для каждого набора правил. Аномалии, возникающие между разными наборами правил, в данной работе не

рассматриваются: сами по себе такие аномалии не нарушают семантику отдельного набора и потому не относятся к критическим ошибкам политики безопасности. Однако проверка высокоуровневых требований к сети и политики безопасности в целом — важная задача, которая требует учёта топологии всей сети, маршрутизации пакетов, а не только правил фильтрации, и потому решается методами анализа достижимости. Такой анализ является самостоятельным направлением и выходит за рамки данной работы.

За основу классификации аномалий между правилами взята модель FIREMAN [2] (см. 3.4). Класс правила определяется не попарным сравнением, а сопоставлением правила с тем, как все предшествующие правила вместе уже обработали трафик. Благодаря этому можно обнаруживать аномалии, возникающие из-за совместного действия нескольких предшествующих правил. Эта модель также была значительно расширена в рамках данной работы.

1. **Выключенные правила (disabled)** — правила, помеченные как выключенные (`isEnabled` ложно), исключаются из основного анализа сразу и относятся к классу `disabled`: их предикаты не участвуют в накоплении эффекта, поскольку на работу устройства они не влияют, однако сам факт наличия выключенного правила является информационным отклонением. В случае ACL, которые не имеют возможности выключаться, правила считаются выключенными, если ACL не имеет точек применения: поле `isEnabled` проставляется на стадии нормализации.
2. **Истекшие правила (expired)** — правило ссылается на расписание с ограничением времени действия, дата окончания которого уже прошла на момент анализа. Такое правило заведомо не сработает ни на одном пакете и подлежит удалению. Тоже не является критической ошибкой и относится к информационным отклонениям. Истёкшие правила также не участвуют в дальнейшем анализе.
3. **Неиспользуемые правила (unused)** — по статистике срабатываний правило давно не применялось: счётчик попаданий равен нулю либо последнее срабатывание (а при его отсутствии — дата изменения или создания) старше заданного порога `daysUntilUnused`. Признак потенциально устаревшего правила; требует подтверждения администратора, также относится к информационным отклонениям.
4. **Затенение (shadowed)** — правило не сработает ни на одном пакете, поскольку весь сопоставляемый им трафик уже обработан предшествующими правилами с противоположным действием. Явная ошибка.
5. **Избыточность (redundant)** — удаление правила не меняет семантику набора: весь его трафик уже обрабатывается предшествующими правилами с тем же действием либо текущее правило само поглощает более раннее.
6. **Обобщение (generalizing)** — правило является надмножеством более раннего правила с противоположным действием. Не обязательно ошибка: часто это осо-

знанный паттерн “разрешить общее, кроме частного”, поэтому относится к предупреждениям.

7. **Корреляция (correlating)** — правило пересекается по части трафика с предшествующим правилом противоположного действия, но ни одно из них не включает другое; результат зависит от порядка правил. Предупреждение.

Правило, не подпавшее ни под один из классов, считается корректным (**normal**).

Отдельно рассматривается *правило по умолчанию* — последнее правило набора с предикатом, сопоставляемым со всем трафиком (например, `deny ip any any`). Такое правило по своей роли обязано поглощать весь трафик, не обработанный предшествующими правилами, поэтому его попадание в классы ошибок не свидетельствует об ошибке конфигурации. Поэтому для этого правила не имеет смысла формировать ошибки, но оно всё ещё участвует при поиске аномалий в остальных правилах.

Каждое обнаруженное отклонение правила описывается не просто меткой класса, а *связью* с конкретным правилом, породившим аномалию. Для пары “анализируемое правило — связанное правило” дополнительно вычисляется отношение по каждому квалификатору: для поля анализируемого правила указывается теоретико-множественное отношение с полем связанного правила. Это позволяет в отчёте показать администратору не только факт аномалии, но и то, как именно анализируемое правило соотносится со связанным.

Ошибки уровня объектов. Помимо самих правил, анализу подвергаются именованные объекты политики — сетевые объекты, группы транспортных портов, сервисные объекты, расписания и другие. Проверки этого уровня имеют смысл лишь для объектов, реально заведённых в конфигурации устройства: сюда относятся как явные объекты (`isExplicit` истинно), так и объекты тех видов, у которых флаг `isExplicit` вообще не предусмотрен. Объекты, синтезированные нормализатором из выражений внутри правил (`isExplicit` ложно), самостоятельными сущностями конфигурации не являются и из анализа этого уровня исключаются. Выделяются классы:

- **Дубликат (duplicated)** — несколько явных объектов одного вида с разными именами, но совпадающим содержимым. Совпадение определяется по нормализованному представлению объекта: его элементы приводятся к каноническому виду и упорядочиваются, поэтому различие лишь в порядке перечисления дубликатом не считается. Это явная ошибка. Дубликаты раздувают конфигурацию и опасны при сопровождении: изменив один объект, администратор легко забывает про его копии, и правила, ссылающиеся на разные имена, начинают расходиться по смыслу. Стоит отметить, что для объектов нельзя ввести более слабый класс ошибки — избыточность, когда один объект является подмножеством другого: это валидный сценарий, при котором сознательно заводят общий объект и его более частный вариант.

- **Непривязанный объект (unattached)** — объект не используется ни одним правилом политики. Предупреждение: такой объект засоряет конфигурацию и затрудняет аудит, пользователь мог забыть прикрепить объект куда хотел или правило было удалено. В правиле не могут быть ссылки на несуществующие объекты — эта проблема решается ещё на стадии сбора и валидации собранных данных.
- **Объект, используемый только неиспользуемыми правилами (unused within rule)** — объект задействован, но исключительно правилами, которые сами отнесены к классу `unused`. Косвенный признак того, что объект потерял актуальность вместе с правилами.

4.3 Алгоритм поиска аномалий между правилами

В предыдущем подразделе были введены классы аномалий уровня правил и объектов. Однако для определения классов `shadowed`, `redundant`, `generalizing`, `correlating` необходимо иметь возможность сравнивать правила между собой. Для этого вводится пространство, в котором можно кодировать предикаты правил и сравнивать их между собой.

Пространство заголовков. Кодирование предиката строится по образцу геометрической модели Header Space Analysis [13]. В этой модели заголовок рассматривается как последовательность из L битов, то есть как точка в пространстве $\mathcal{H} = \{0,1\}^L$, называемом *пространством заголовков*. Конкретный пакет — точка этого пространства, а любое множество пакетов — его область.

Пространство $\mathcal{H}$ объединяет все поля, по которым правила сопоставляют трафик. Перечень этих полей и их разрядности задаёт *формат*. Биты пространства делятся между полями по порядку: каждое поле занимает свой непрерывный участок длиной в его разрядность, и участки идут друг за другом. Например, при полях “протокол” (8 битов), “адрес отправителя” и “адрес получателя” (по 32 бита) первые 8 битов вектора отвечают протоколу, следующие 32 — адресу отправителя, ещё 32 — адресу получателя.

В формат входят квалификаторы правила, по которым оно сопоставляет отдельный пакет: адрес отправителя и получателя, IP-протокол, порты отправителя и получателя, тип и код ICMP, флаги TCP, состояние соединения, а также непрозрачные квалификаторы прикладного уровня и контекста соединения — приложение, URL-категория, пользователь, устройство, зона, метка соединения. Квалификаторы, относящиеся не к отдельному пакету, а к правилу в целом, — расписание — в пространство $\mathcal{H}$ не входят и учитываются отдельно (см. класс `expired` в 4.2). Часть полей формата отвечает реальным битовым полям заголовка пакета (адреса, протокол, порты, тип и код, флаги), часть — непрозрачным символьным квалификаторам, не имеющим битового представления в пакете; то, как кодируются и те и другие, описано ниже. Существенно, что единое пространство $\mathcal{H}$ вмещает и битовые поля, и символьные квалификаторы, и над всеми ними применимы одни и те же операции.

Кодирование предиката. Простейшая область в $\mathcal{H}$ задаётся *wildcard-выражением* — последовательностью из L символов, где каждый символ есть 0, 1 или * (произвольный бит). Wildcard-выражение задаёт множество всех точек пространства $\mathcal{H}$, совпадающих с ним во всех позициях, отличных от *; геометрически такое множество — гиперкуб пространства заголовков. Одним wildcard-выражением, заданным на участке битов поля, описывается лишь простейшее условие правила на это поле. Пример — подсеть 192.168.1.0/24: ей отвечает wildcard-выражение, в котором 24 старших бита адреса фиксированы, а 8 младших произвольны, то есть гиперкуб всех адресов подсети. Но условие на поле в общем случае сложнее и одним гиперкубом не описывается: произвольный диапазон портов или интервал адресов “начало–конец”, не выражаемый одной маской, а тем более условие, ссылающееся на составной объект из нескольких компонентов, задаёт область, которая является *объединением* нескольких гиперкубов. Поэтому область любого поля представляется набором wildcard-выражений, а одиночное выражение — лишь его частный случай.

Каждой битовой координате пространства сопоставлена булева переменная x_i . Предикат wildcard-выражения — конъюнкция значимых позиций: бит 1 даёт x_i , бит 0 — отрицание $\bar{x}_i$, позиции * в конъюнкцию не входят. Область из нескольких wildcard-выражений кодируется дизъюнкцией их предикатов. Таким образом, предикат области любого поля можно представить в виде дизъюнктивной нормальной формы (ДНФ).

Кодирование объектов модели. Правило не хранит значения квалификаторов непосредственно, а ссылается по имени на объекты политики безопасности (см. 4.1.3), поэтому предикат поля строится по объекту, на который поле ссылается. Объект может перечислять элементарные компоненты непосредственно и одновременно ссылаться на другие объекты того же вида, образуя группу. Поэтому объект сначала *раскрывается* — рекурсивным разрешением вложенных ссылок — в плоское множество элементарных компонентов, а затем кодируется покомпонентно. Также проверяется, что циклические ссылки в конфигурации устройств не были допущены.

Сетевой объект задаёт условие на адрес отправителя или получателя. Его раскрытие даёт множество адресных компонентов $\{c_1, \dots, c_m\}$, каждый из которых задаёт область пространства адресов: компонент вида “адрес–маска” — один гиперкуб; компонент вида “начало–конец” — произвольный интервал адресов, который представляется объединением гиперкубов (выровненных адресных блоков), покрывающих этот интервал. Каждому компоненту c_i отвечает предикат C_i , а флаг `isNegated` этого компонента отмечает его как *отрицаемый*. Обозначим через C^+ объединение компонентов с `isNegated = false` (положительных), а через C^- — объединение компонентов с `isNegated = true` (отрицаемых):

$$C^+ = \bigvee_{i: \text{isNegated}(c_i)=\text{false}} C_i, \quad C^- = \bigvee_{i: \text{isNegated}(c_i)=\text{true}} C_i.$$

Предикат поля адреса — это базовое множество адресов, из которого исключён C^- :

$$B \wedge \overline{C^-}, \quad B = \begin{cases} C^+, & \text{если есть хотя бы один компонент с } \text{isNegated} = \text{false}, \\ \Omega, & \text{иначе.} \end{cases}$$

Здесь Ω — всё пространство адресов. Случай $B = \Omega$ отвечает объекту вида “любой адрес, кроме перечисленных”: если все компоненты имеют $\text{isNegated} = \text{true}$, отрицаемые исключаются не из пустого множества, а из всего пространства. Незаданный сетевой объект (**any**) задаёт по полю адреса всё пространство значений Ω — тождественно истинный предикат.

Сервис кодируется сложнее, так как описывает сразу несколько полей — протокол и, при их наличии у протокола, порты или тип и код. Сервисный объект, как и сетевой, раскрывается рекурсивно в плоский список *транспортов*, каждый из которых задаёт один IP-протокол со своими параметрами. Предикат отдельного транспорта — конъюнкция предикатов его полей: предиката протокола, предиката портов отправителя и предиката портов получателя (для TCP и UDP) либо предикатов типа и кода ICMP (для ICMP; их кодирование описано ниже). Условие на порты транспорт задаёт не значением, а ссылкой на группу транспортных портов — множество интервалов номеров портов; каждый интервал кодируется набором wildcard-выражений, а предикат поля порта — их дизъюнкция. Предикат всего сервиса — дизъюнкция предикатов транспортов: пакет совпадает с сервисом, если он совпадает хотя бы с одним из его транспортов. Незаданное поле транспорта трактуется как **any**: например, транспорт с указанным протоколом, но без портов сопоставляется с любым портом. Так составной сервис, объединяющий, скажем, TCP- и UDP-транспорт под одним именем, даёт предикат, истинный на заголовках обоих протоколов.

Числовые поля заголовка — тип и код ICMP, поле TTL — кодируются так же, как порты: условие задаётся набором интервалов значений, каждый интервал представляется объединением гиперкубов, а предикат поля — их дизъюнкцией. Одиночное значение (например, конкретный тип ICMP) — частный случай интервала с совпадающими концами.

Флаги TCP занимают в формате поле, каждый бит которого отвечает одному флагу (SYN, ACK, FIN, RST и т. д.). Условие правила перечисляет флаги, которые должны быть установлены, и флаги, которые должны быть сброшены; биты установленных флагов фиксируются единицей, сброшенных — нулём, а флаги, не упомянутые в условии, остаются произвольными. При модификаторе *match_all* (совпадение требуется по всем перечисленным флагам сразу) предикат — одно wildcard-выражение с указанной комбинацией фиксированных битов; при модификаторе *match_any* (достаточно хотя бы одного из перечисленных установленных флагов) предикат — дизъюнкция wildcard-выражений, по одному на каждый требуемый флаг.

Категориальные поля — состояние соединения, а также непрозрачные квалификаторы: приложение, URL-категория, пользователь, устройство, зона, метка соедине-

ния. Их значения не имеют битового представления в пакете, поэтому каждому различному значению, встретившемуся в наборе правил, заранее присвоен числовой код, а само поле занимает в формате $\lceil \log_2 k \rceil$ битов (k — число различных значений). Условие правила на категориальное поле — это множество допустимых значений (одно или несколько имён приложений, состояний соединения и т. п.); каждое значение задаётся одним wildcard-выражением, фиксирующим биты его кода, а предикат поля — дизъюнкция таких выражений по всем допустимым значениям. Незаданное категориальное поле, как и прочие незаданные квалификаторы, даёт всё пространство значений.

Сборка предиката правила. Предикат P всего правила собирается из предикатов полей. Каждый предикат поля строится на отведённом этому полю участке битов, после чего предикаты всех полей объединяются конъюнкцией: правило сопоставляется с пакетом тогда и только тогда, когда одновременно совпадают все его поля. В терминах пространства заголовков P — прямое произведение областей отдельных полей. Незаданный квалификатор (**any**) даёт по своему полю всё пространство значений, то есть тождественно истинный предикат, и в конъюнкцию ограничений не вносит. Чтобы выполнять над предикатами теоретико-множественные операции, к которым сводится анализ аномалий, — пересечение, объединение, дополнение, проверку включения, — предикат представляется бинарной решающей диаграммой (BDD): она даёт каноническое представление булевой функции, на котором все эти операции выполняются как стандартные операции над диаграммами. Это же представление предиката используется и анализом достижимости, поэтому формат заголовка и кодирование правила в предикат для двух видов анализа общие.

Деление набора правил на подгруппы. Формат, описывающий сразу все возможные поля, избыточен для любого конкретного правила: не всякое поле осмысленно для всякого пакета. Номера портов отправителя и получателя есть только у TCP и UDP, флаги — только у TCP, тип и код — только у ICMP; IPv4- и IPv6-адреса в одном пакете не встречаются. Если кодировать все правила в едином пространстве $\mathcal{H}$ полного формата, переменные неприменимого поля остаются в предикате правила произвольными: предикат `permit icmp any any` не зависит от 32 битов портов, но эти биты всё равно присутствуют в общем пространстве, и анализ оперирует пространством заведомо большей размерности, чем требуется. Так как все классы аномалий проверяются булевыми операциями над предикатами, а стоимость этих операций растёт с размерностью пространства, набор правил перед анализом разбивается на *подгруппы* с однородной структурой пакета, и каждая анализируется в своём, сокращённом пространстве заголовков.

Разбиение пространства. В основе разбиения лежит наблюдение: применимость одних полей зависит от значения других. Из приведённых выше примеров видно, что именно значение протокола определяет, какие из остальных полей вообще участвуют в сопоставлении пакета. Поля, значение которых так задаёт применимость остальных,

назовём *ключевыми*; в рассматриваемом случае ключевые поля — IP-протокол и версия IP.

Разбиение строится по ключевым полям. Каждому набору их конкретных значений отвечает своя *область* пространства заголовков $\mathcal{H}$ — множество всех пакетов с такими значениями ключевых полей (например, “все ТСП-пакеты версии IPv4”). Эти области образуют *разбиение* $\{H_1, \dots, H_n\}$: они попарно не пересекаются ($H_i \cap H_j = \emptyset$ при $i \neq j$), так как пакет не может иметь двух разных протоколов, и в объединении покрывают всё пространство ($\bigcup_i H_i = \mathcal{H}$). Подгруппа — это область H_g вместе с *сокращённым форматом*: ключевые поля в нём зафиксированы значениями области, а поля, неприменимые при этих значениях, из формата исключены (для области ТСП в формате есть порты и флаги, но нет типа и кода ICMP). Тем самым размерность пространства, в котором ведётся анализ подгруппы, сведена к числу полей, реально применимых к её трафику. Набор ключевых полей — параметр анализа: чем больше полей в него включено, тем мельче разбиение и сильнее сокращение формата, но тем больше получается подгрупп.

Используемое разбиение. По ключевым полям — версии IP и IP-протоколу — разбиение строится двухуровневым. Верхний уровень — *версия IP* (IPv4 либо IPv6): она определяет разрядность самых широких полей формата, адресов отправителя и получателя. Нижний уровень — *группа протоколов* внутри версии: `tcp`, `udp`, `icmp` и собирательная группа `other` для всех прочих IP-протоколов (GRE, ESP, OSPF и т. п.), у которых нет ни портов, ни типа с кодом. Подгруппа — пара $g = \langle \text{версия}, \text{группа} \rangle$. Сокращённый формат подгруппы содержит адреса (разрядности, отвечающей версии), поле протокола и категориальные измерения — во всех подгруппах; порты отправителя и получателя — в `tcp` и `udp`; флаги — в `tcp`; тип и код — в `icmp`; формат группы `other` ни портов, ни флагов, ни типа с кодом не содержит. Это разбиение — не единственно возможное: при необходимости его можно как огрубить (например, не делить по протоколу), так и уточнить, добавив ключевые поля; алгоритм анализа от конкретного разбиения не зависит и требует от него лишь свойств непересекаемости и покрытия.

Отнесение правила к подгруппам. Правило приписывается ко всем подгруппам, область которых пересекается с предикатом правила; в каждой такой подгруппе правило представляется своей *проекцией* — предикатом, построенным по сокращённому формату этой подгруппы. Принадлежность определяется по условиям правила на ключевые поля: правило относится к подгруппе g , если его условие на каждое ключевое поле совместно с тем значением, которым это поле зафиксировано в H_g . Незаданное условие (`any`) совместно с любым значением, поэтому правило без ограничения на ключевое поле попадает во все подгруппы по этому полю. Для используемого разбиения это означает: версия IP — по адресным компонентам сетевых объектов отправителя и получателя (правило с компонентами только одной версии относится к ней одной, с `any` — адресом или компонентами обеих версий — к обеим); группа протоколов — по транспортным сервисным объектам (каждый транспорт несёт номер IP-протокола и относит пра-

вило к соответствующей группе, сервис из нескольких транспортов разносит правило по нескольким группам, **any**-сервис — по всем четырём). Итоговое множество подгрупп правила — декартово произведение его версий IP и групп протоколов: правило **permit ip any any** попадает во все восемь подгрупп, правило с конкретным IPv4-адресом и сервисом “DNS” (транспорты TCP/53 и UDP/53) — в $\langle \text{IPv4, tcp} \rangle$ и $\langle \text{IPv4, udp} \rangle$, а правило с IPv4-адресом и протоколом ICMP — ровно в одну. При построении проекции на подгруппу g квалификаторы, не входящие в её сокращённый формат, отбрасываются, а квалификаторы, относящиеся к другим областям разбиения, в предикат не вносятся: в проекции на $\langle \text{IPv4, udp} \rangle$ от сервиса “DNS” остаётся только UDP-транспорт. Правило, чья проекция на подгруппу пуста, из этой подгруппы исключается.

Внутри каждой подгруппы, состоящей хотя бы из 2 правил, выполняется самостоятельный анализ: правила подгруппы упорядочиваются по приоритету, и к ним применяется описанный ниже алгоритм классификации. Подгруппы анализируются независимо и поддаются параллельному выполнению. Результаты затем объединяются: *Затенение* и *избыточность* — это утверждение о том, что *весь* трафик правила уже был обработан другими правилами; оно справедливо для правила в целом, только если справедливо для каждой его части, поэтому такая аномалия объявляется, только если получена во *всех* подгруппах правила. Если хотя бы в одной подгруппе правило срабатывает, оно не затенено и не избыточно. *Корреляция* и *обобщение* — это утверждение о том, что у правила есть пересечение с другим правилом; оно справедливо для правила в целом, как только справедливо хотя бы для одной его части, поэтому такая аномалия объявляется, если получена хотя бы в *одной* подгруппе. Построенные связи правил объединяются по всем подгруппам.

Корректность опирается на два свойства разбиения — непересекаемость областей и покрытие ими всего пространства, — в силу которых каждый пакет принадлежит ровно одной подгруппе. Поэтому предикат правила в полном пространстве $\mathcal{H}$ есть дизъюнкция его проекций по подгруппам, а всякое теоретико-множественное соотношение между двумя правилами (включение, пересечение, дополнение) распадается на независимые соотношения внутри подгрупп. Анализ по подгруппам даёт тот же результат, что и анализ в едином пространстве, но в пространствах меньшей размерности и без переменных, не относящихся к трафику подгруппы.

Также если внутри конкретной подгруппы правилами не используются какие-либо квалификаторы, то их можно исключить из формата подгруппы, чтобы ещё больше сократить пространство заголовков.

Поиск аномалий. Здесь и далее будем работать в рамках одной подгруппы; для остальных подгрупп логика анализа повторяется независимо. В основе поиска аномалий лежит алгоритм FIREMAN, подробно разобранный в обзоре (см. 3.5.3): правила подгруппы обходятся по приоритету, по мере обхода поддерживается накопленное состояние $\langle A, D, F \rangle$ — множества уже разрешённых, запрещённых и уведённых на другой путь пакетов, а также производное множество $R = \overline{A \cup D \cup F}$ ещё не обработанных па-

кетов, — и аномалии очередного правила $\langle P_j, a_j \rangle$ определяются тем, как его предикат P_j расположен относительно R , A и D . Аномалия $\langle P_j, a_j \rangle$ фиксируется не только меткой, а ещё и *связью* с конкретным предшествующим правилом, её породившим. Для пары “проверяемое правило — связанное” дополнительно вычисляется отношение по каждому полю: предикаты обоих правил проецируются на отдельные поля, проекции переводятся в интервалы значений, и для каждого поля определяется теоретико-множественное отношение (совпадение, надмножество, подмножество, пересечение). Это даёт в отчёте не только факт аномалии, но и то, по каким полям и как анализируемое правило соотносится со связанным.

Базовый алгоритм исходит из того, что предикат P_j *точно* описывает множество сопоставляемых правилом заголовков. Но это выполняется не всегда. Пространство заголовков $\mathcal{H}$ имеет фиксированную структуру и описывает лишь заранее заданный набор квалификаторов; всё многообразие условий сопоставления, которые поддерживают реальные устройства, оно вместить не может. Если правило опирается на квалификатор, которому в $\mathcal{H}$ нет измерения, этот квалификатор в предикате P_j выразить нечем — и при нормализации он остаётся нераспознанным: у такого правила поле `isComplete` ложно, а исходные фрагменты условия сохранены в `unknownMatches` (см. 4.1, “Полнота нормализации”). Предикат P_j такого *неполного* правила построен лишь по той части условия, которая в $\mathcal{H}$ выразима, и потому может в действительности описывать как более узкое множество, так и более широкое.

Из-за этого, например, затенение, обнаруженное в пространстве $\mathcal{H}$, может не быть затенением в действительности. Покажем это на конфигурации iptables:

```
iptables -A FORWARD -d 10.0.0.10 -p tcp --dport 80 \
    -m string --string "/admin" --algo bm -j DROP
iptables -A FORWARD -d 10.0.0.10 -p tcp --dport 80 -j ACCEPT
```

Первое правило блокирует HTTP-запросы к узлу 10.0.0.10, в payload которых встречается подстрока `/admin`; второе — разрешает остальной HTTP-трафик к этому узлу. Это осмысленная пара правил, и второе из них реально работает. Однако условие `-m string` — поиск подстроки в содержимом пакета — в пространстве $\mathcal{H}$ невыразимо: измерения “подстрока в полезной нагрузке пакета” в формате заголовка нет. При нормализации это условие отбрасывается, и предикат первого правила строится лишь по выразимой части — `dst=10.0.0.10, proto=tcp, dport=80`. Но ровно такой же предикат и у второго правила. В $\mathcal{H}$ первое правило выглядит точной копией второго, и анализатор объявит второе правило затенённым первым. На самом деле затенения нет: первое правило сопоставляет лишь узкий класс запросов, а второе обрабатывает весь остальной трафик к узлу. Затенение в огрублённом пространстве оказалось ложным — его породил квалификатор, которого в $\mathcal{H}$ нет. Поэтому подставлять P_j неполного правила в анализ напрямую нельзя.

Источник ошибки в том, что для неполного правила истинность нераспознанной части на конкретном пакете анализатору *неизвестна* — это не “истина” и не “ложь”.

Представим предикат неполного правила в виде конъюнкции $P_j = P_j^{\text{зн}} \wedge U$, где $P_j^{\text{зн}}$ — предикат, построенный по *распознанной* части квалификаторов, а U — предикат *нераспознанной* части, неизвестный анализатору как целое. Точное множество заголовков правила зависит от U , но про U известно лишь, что это какое-то подмножество пространства заголовков: $\emptyset \subseteq U \subseteq \Omega$. Отсюда сразу следуют границы и для всего предиката правила:

$$\emptyset \subseteq P_j \subseteq P_j^{\text{зн}},$$

поскольку $P_j^{\text{зн}} \wedge \emptyset = \emptyset$, а $P_j^{\text{зн}} \wedge \Omega = P_j^{\text{зн}}$.

Эти две границы и берутся за две оценки предиката неполного правила:

- *нижняя оценка* $P_j^- = \emptyset$ — нераспознанная часть считается *не сопоставляющей ни одного пакета* ($U = \emptyset$): правило не сопоставляет ничего;
- *верхняя оценка* $P_j^+ = P_j^{\text{зн}}$ — нераспознанная часть считается *сопоставляющей все пакеты* ($U = \Omega$): правило сопоставляет всё, что допускает распознанная часть.

Для полного правила U известно и обе оценки совпадают с P_j ; расходятся они только на неполных правилах.

Чтобы по этим оценкам судить о поведении *всего набора*, анализ ведётся по образцу формально верифицированного анализа iptables Дикманна [5]: вместо одного набора с неопределённостью строятся два полных набора — *разрешающее* и *запрещающее замыкание*, — в каждом из которых неполнота уже доопределена, и каждый обходится обычным алгоритмом с обычным обновлением множеств A , D , F и R . Какую оценку подставить вместо P_j неполного правила, зависит от его действия: замыкание доопределяет неполноту *в пользу* разрешения или запрета, а не предиката самого по себе. В *разрешающем* замыкании неполное правило `allow` берётся по верхней оценке P_j^+ , а неполное `deny` — по нижней оценке P_j^- : так набор разрешает *не меньше* трафика, чем настоящий. В *запрещающем* замыкании — наоборот: неполное `allow` по нижней оценке P_j^- , неполное `deny` по верхней оценке P_j^+ , и набор разрешает *не больше* настоящего. Полные правила в обоих замыканиях одинаковы. Настоящее поведение набора заключено между двумя замыканиями, и это позволяет искать аномалии на неполных правилах *консервативно* — так, чтобы ни одна выданная критическая ошибка не оказалась ложной.

Затенение, избыточность и обобщение требуют, чтобы предикат проверяемого правила был достаточно *широк*: затенение и избыточность — это утверждение “весь трафик правила перекрыт предшествующими правилами”, а обобщение — “правило содержит более раннее правило” ($P_x \subseteq P_j$); и включение P_j в множество перекрытий, и включение чужого предиката в P_j тем легче, чем P_j шире. Чтобы такое утверждение было *гарантированным* независимо от нераспознанной части, проверяемое неполное правило берётся по его *широчайшей* форме — верхней оценке $P_j^+ = P_j^{\text{зн}}$: если аномалия налицо даже для самого широкого из возможных предикатов, то она налицо и для настоящего, $P_j \subseteq P_j^+$, какой бы ни оказалась нераспознанная часть. Какое замыкание

даёт правилу эту форму, зависит от его действия: неполное правило **allow** взято по P_j^+ в *разрешающем* замыкании, неполное **deny** — в *запрещающем*. Поэтому затенение, избыточность и обобщение **allow**-правила проверяются в разрешающем замыкании, а **deny**-правила — в запрещающем; противоположное замыкание для этой проверки бесполезно: в нём проверяемое правило взято по нижней оценке $P_j^- = \emptyset$, а пустое множество тривиально “перекрыто” любым предшествующим правилом и не содержит ни одного чужого предиката, так что проверка давала бы ложный результат для каждого неполного правила. Критическая ошибка (**shadowed** или **redundant**) объявляется только тогда, когда поиск аномалий даёт её в замыкании, где правило взято по P_j^+ .

Корреляция проверяется *зеркально*. Это утверждение другого рода: не о включении, а о *пересечении* — “у правила есть общий трафик с правилом противоположного действия” ($P_j \cap N \neq \emptyset$), причём ни одно из правил не включает другое. Пересечение тем труднее обеспечить, чем P_j *уже*; поэтому, чтобы корреляция была *гарантированной*, неполное правило берётся по его *наименьшей* форме — нижней оценке P_j^- : если пересечение есть даже у самого узкого предиката, оно есть и у настоящего, $P_j \supseteq P_j^-$. Замыкание, дающее правилу эту форму, снова зависит от действия: неполное **allow** взято по P_j^- в *запрещающем* замыкании, неполное **deny** — в *разрешающем*. Противоположное замыкание для корреляции бесполезно: в нём предикат расширен до $P_j^{\text{зн}}$, что может породить пересечение, которого у настоящего правила нет, — ровно так возникала ложная аномалия в примере с **-m string**.

Аномалия считается подтвержденной, если она была найдена в подходящем для неё замыкании (разрешающем или запрещающем, по правилам выше): тогда она проявляется при любом трафике, независимо от того, чтобы было написано в нераспознанной части. Аномалия была не подтверждена в подходящем замыкании, но обнаружена в другом замыкании — лишь *возможна*: администратор может оценить конфликт самостоятельно. Тем самым анализ неполных правил не теряет ни одной реальной аномалии и не порождает ложных критических ошибок — ценой того, что часть выводов по таким правилам формулируется как предупреждение, а не как ошибка.

5 Описание практической части

5.1 Интеграция в систему инвентаризации и валидации сетевых инфраструктур

В рамках работы разработан прототип системы анализа правил фильтрации и объектов политики безопасности — *RuleAnalyzer*, и был интегрирован в систему инвентаризации и валидации сетевых инфраструктур, разрабатываемую в ООО «Дипси Нетворкс». Эта система собирает данные о сетевых устройствах, отслеживает их изменения и проверяет на соответствие наборов требований; анализ правил фильтрации и объектов политики безопасности — одна из таких проверок, и анализатор был встроен в систему в качестве отдельного модуля валидации.

Примерная архитектура модулей системы инвентаризации и валидации сетевых инфраструктур приведена на рис. 1;

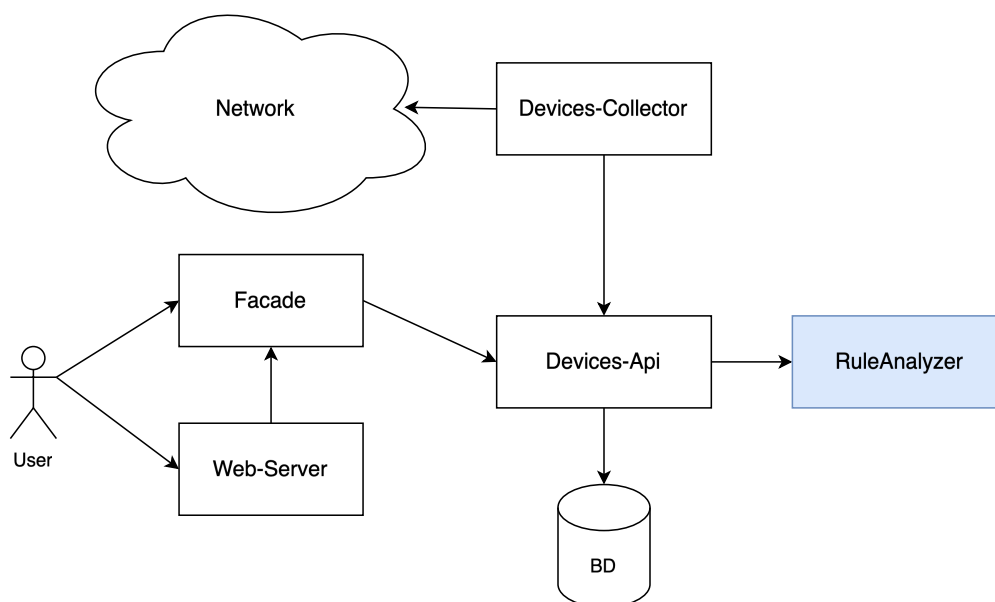


Рисунок 1 – Схема обработки конфигурации устройства

Система состоит из следующих модулей:

- **Web-Server** и **Facade** — точки входа пользователя в систему. **Web-Server** отдаёт веб-интерфейс, **Facade** принимает API запросы от пользователя и маршрутизирует их к внутренним модулям, скрывая их устройство от клиента.
- **Devices-Collector** — модуль сбора данных о сетевых устройствах. Подключается к сетевым устройствам, выполняет на них команды и забирает вывод команд в исходном текстовом формате производителя, затем выполняет разбор и нормализацию выводов, приводя к унифицированной модели сетевого устройства.
- **Devices-API** — центральный модуль. Принимает собранные нормализованные данные от **Devices-Collector**, сохраняет их в базе данных (BD), обслуживает

запросы от **Facade**. При получении нового снимка сети, отправляет полученные данные на анализ в **RuleAnalyzer**, и сохраняет результаты анализа в базу данных.

- **RuleAnalyzer** — анализатор правил фильтрации и объектов политики безопасности, описанный и реализованный в данной работе (выделен синим цветом).

Для интегрирования нового модуля в систему, также потребовались значительные доработки остальных модулей системы — каждый из них был адаптирован под новый функционал: была расширена модель данных сетевого устройства (см. 4.1), чтобы она могла содержать данные о правилах фильтрации и объектах политики безопасности. **Devices-Collector** был адаптирован под особенности сбора правил фильтрации разных платформ производителей. В **Devices-API** было добавлено сохранение модели политики и отчётов анализа в базу данных, а в **Web-Server** — представление отчёта об аномалиях в веб-интерфейсе.

Модуль анализа реализован на языке Go — том же, на котором написана остальная бэкенд часть системы, что упрощает интеграцию. Ниже подробно рассмотрены сбор и нормализация выводов команд и результаты нагрузочного тестирования **RuleAnalyzer**.

5.2 Сбор данных с устройств и их нормализация

Входными данными алгоритма анализа служит унифицированная модель политики безопасности (см. 4.1), а не сырые выводы команд устройства. Задача коллектора — получить заполненную нормализованную модель сетевого устройства по набору таких выводов. Прямое решение этой задачи затруднено тем, что синтаксис выводов сильно зависит от платформы и даже от версии операционной системы. Остальные модули системы получают уже унифицированное представление и не зависят от конкретного производителя оборудования, чтобы позволяет работать единообразно с любым сетевым устройством.

Этапы обработки. Обработка отдельного сетевого устройства на коллекторе состоит из нескольких этапов:

1. **Сбор сырого вывода.** Коллектор подключается к устройству по SSH или Telnet и выполняет набор команд, заданный для платформы. Команды и их состав версионированы: для каждой пары “операционная система — версия” хранится свой список. Результат каждой команды — сырой текстовый вывод CLI. Текстовый CLI выбран в качестве источника данных как единственный универсально доступный интерфейс: NETCONF, gNMI, REST либо не реализованы для нужных команд, либо доступны не на всех версиях операционной системы поддерживаемых платформ.
2. **Разбор вывода шаблоном TextFSM.** Вывод команды представляет из себя не структурированные данные, а текст, рассчитанный на чтение человеком: таблицы

с фиксированными колонками, отступы, многострочные записи, повторяющиеся блоки с заголовками. Чтобы дальнейшие этапы работали не с произвольным текстом необходим этап преобразования текста в структурированные записи.

В качестве инструмента такого преобразования выбран *TextFSM* — де-факто стандарт для разбора вывода сетевых CLI, изначально разработанный в Google и широко применяемый в библиотеках сетевой автоматизации (*ntc-templates*, *Netmiko*, *NAPALM*). Шаблон *TextFSM* представляет собой описание конечного автомата: задаётся набор именованных *состояний*, для каждого — регулярные выражения, сопоставляющиеся со строками вывода, и действия — какие значения сохранять в поля и в какое состояние переходить. Такая модель удобна именно для построчного табличного текста или текста с большим количеством ключевых слов.

Выбор *TextFSM* обоснован тремя причинами. Во-первых, формат шаблонов декларативный и отделён от кода: добавление поддержки новой команды или новой версии операционной системы сводится к добавлению или правке текстового файла, а не к изменению исходников коллектора. Во-вторых, существует крупная открытая база готовых шаблонов *ntc-templates*, покрывающая значительную часть команд распространённых платформ; собственные шаблоны пишутся лишь для специфичных команд.

Альтернативные подходы рассматривались и были отвергнуты. Разбор регулярными выражениями напрямую, без модели состояний, плохо масштабируется на многострочные записи и быстро теряет читаемость. Написание парсера для каждой команды вручную дорого в сопровождении и не даёт такой гибкости подстройки для конкретной инфраструктуры. Полноценный грамматический разбор всего вывода (через лексер и парсер) на этом уровне избыточен: структура вывода большинства команд проста и хорошо ложится на конечный автомат, а тяжёлый аппарат грамматик целесообразно применять только там, где он действительно нужен — внутри отдельных строк со сложным синтаксисом. Этот этап был реализован на основе *TextFSM* шаблонов.

На выходе этого этапа вывод каждой команды превращается в список записей, где каждая запись — набор полей “имя — значение”; часть значений уже атомарна (числа, имена), часть остаётся строками, требующими дальнейшего разбора.

3. **Грамматический разбор строк правил.** *TextFSM* удобно *режет* вывод на записи и табличные поля, но плохо справляется с семантикой *внутри* одной строки. Строка правил ACL имеют большую гибкость в описаний условий, и эти условия могут не имеют ключевых слов за которые можно было бы зацепиться регулярными выражениями, поэтому в *TextFSM* необходимо описывать все возможные варианты синтаксиса строки правила набором регулярных выражений, что неудобно, неэффективно и непрозрачно. Если регулярные выражения не подойдут, узнать о ошибке очень сложно, так как *TextFSM* не предоставляет информации о том,

какая строка не подошла под шаблон. Поэтому был добавлен дополнительный этап обработки - грамматический разбор. На выходе этого этапа формат записи результата сохраняется.

4. **Сопоставление с моделью.** После грамматического разбора записи имеют общий набор полей, поэтому сборка модели разделена на вендоронезависимую и вендорозависимую части. *Вендоронезависимая часть* едина для всех платформ: нормализует маски и подсети, раскрывает диапазоны портов, связывает правила с объектами по имени и тд. *Вендорозависимые части* вынесены в адаптеры (по одному на платформу) и описывают то, что не сводится к общей форме: семантику специфичных квалификаторов, нормализацию действий и тд. Благодаря такому разделению добавление новой платформы — это одна грамматика и один адаптер, а изменения в модели делаются один раз в общей части.

Этапы 2-4 вместе составляют *нормализацию* — приведение разнородного вывода устройств к единому структурированному представлению, поверх которого уже работает алгоритм анализа.

Грамматический разбор строк правил. Доразбор строковых полей реализован через формальные грамматики. Для каждого вида разбираемой строки — правила ACL, описания именованного объекта и т. п. — описывается грамматика в виде двух частей:

- *лексический анализатор* (сканер) — разбивает строку на поток токенов: числа, IP-адреса, ключевые слова (`permit`, `tcp`, `eq`, ...), идентификаторы, знаки пунктуации.
- *синтаксический анализатор* (парсер) — по продукциям грамматики сворачивает поток токенов в структурированную запись: заполняет именованные поля результата — номер, действие, протокол, адреса с масками, порты и тд.

Лексический и синтаксический анализаторы не пишутся вручную, а *генерируются* по формальному описанию грамматики инструментами `golex` и `goyacc` — реализациями классических генераторов `lex` и `yacc` для языка Go. Грамматика описывается декларативно: для лексера — регулярными выражениями, сопоставляющими фрагменты текста с типами токенов, для парсера — продукциями, описывающими допустимые последовательности токенов и действия по сборке результата. Такой подход даёт два преимущества перед разбором вручную. Во-первых, грамматика проверяема: генератор парсера обнаруживает в ней неоднозначности (конфликты вида `shift/reduce`) ещё на этапе генерации. Во-вторых, поддержка новой платформы или нового диалекта синтаксиса сводится к добавлению отдельной грамматики, не затрагивая разбор остальных платформ.

5.3 Нагрузочное тестирование

Чтобы оценить применимость прототипа к большим наборам правил, проведено нагрузочное тестирование модуля анализа. Тестирование измеряет, как время работы и потребление памяти зависят от размера набора правил.

Методика. Для теста синтезировались наборы правил заданного размера — от 100 до 100 000 правил. Правила генерировались с разными сетевыми объектами и использовался конечный набор сервисов. Размер набора — не единственный параметр, влияющий на стоимость анализа: основной вклад в работу вносит число аномалий, а их число зависит от того, насколько часто правила пересекаются. Поэтому наборы синтезировались с разной долей аномальных правил — 0%, 3%, 5% и 10%: при 0% правила не пересекаются, при 10% доля пересекающихся правил высока. Генерировались аномалии двух типов: затенение и избыточность. Для используемой библиотеки BDD были сняты ограничения по памяти. Для каждой пары “размер набора — доля аномалий” измерялись время анализа и пиковое потребление памяти.

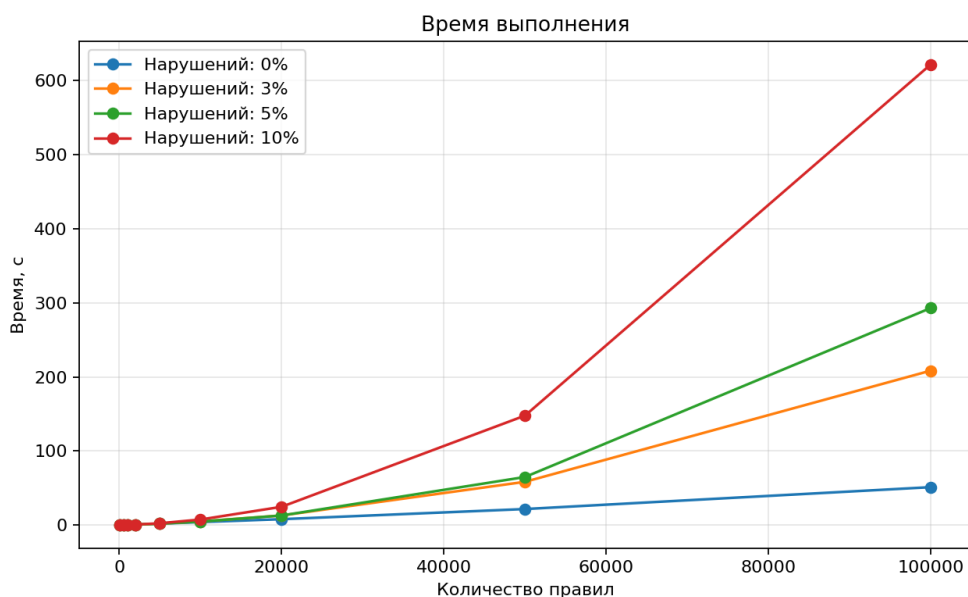


Рисунок 2 – Время анализа в зависимости от размера набора правил

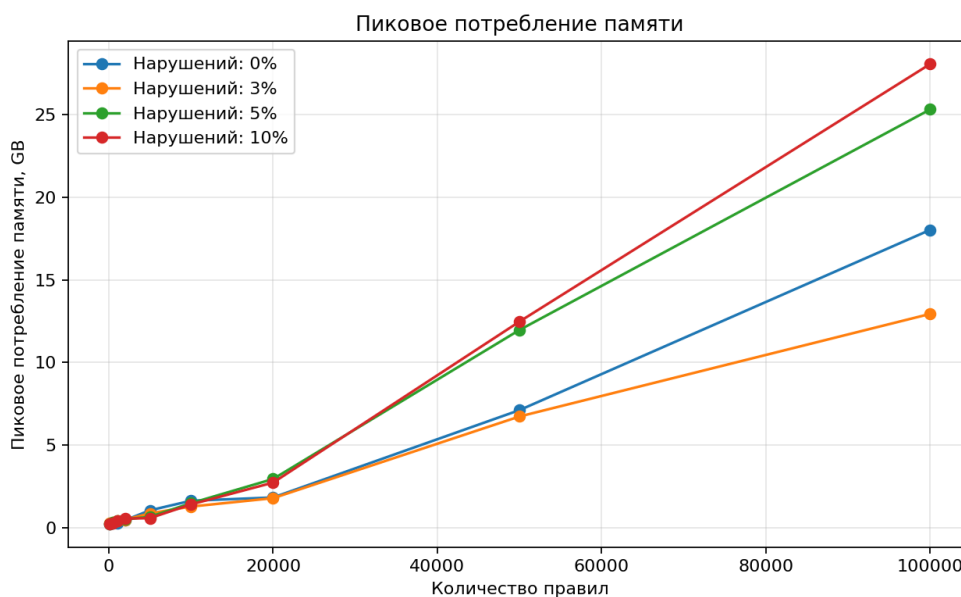


Рисунок 3 – Пиковое потребление памяти в зависимости от размера набора правил

Результаты. На рис. 2 приведено время анализа. На наборах до 20 000 правил анализ занимает единицы и первые десятки секунд при любой доле аномалий. На наибольшем наборе в 100 000 правил время существенно зависит от доли аномалий: от примерно 50 секунд при 0% до примерно 620 секунд при 10%. Такое расхождение ожидаемо: сам обход правил линейный по их числу, а дополнительное время уходит на построение связи правил для найденной аномалии, которых тем больше, чем чаще правила пересекаются.

Пиковое потребление памяти (рис. 3) растёт с числом правил приблизительно линейно и на наборе в 100 000 правил составляет от 13 до 28 ГБ в зависимости от доли аномалий. Память в основном расходуется на хранение предикатов правил и накопленных множеств.

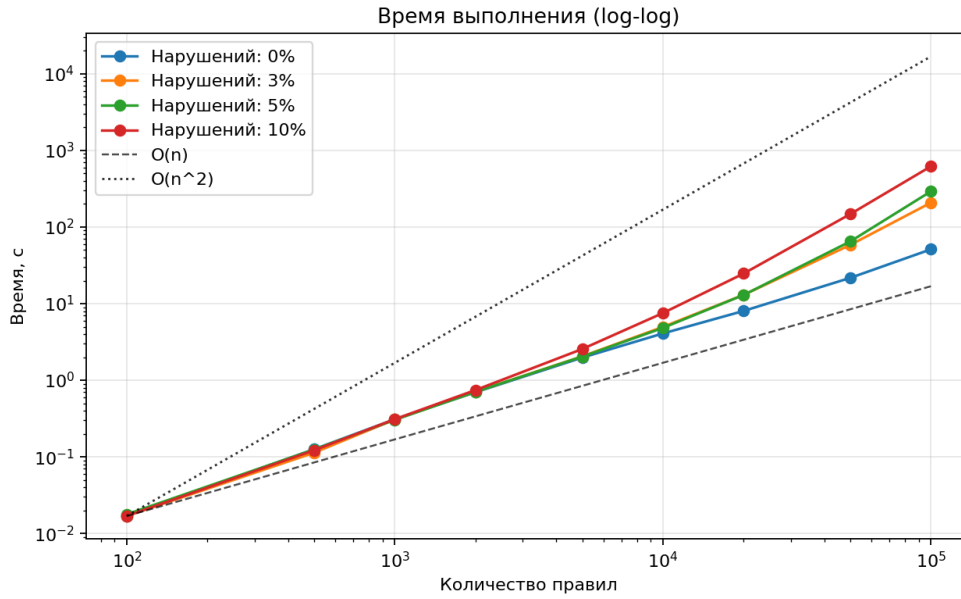


Рисунок 4 – Время анализа в двойном логарифмическом масштабе с опорными кривыми $O(n)$ и $O(n^2)$

Чтобы оценить асимптотику, то же время анализа представлено в двойном логарифмическом масштабе (рис. 4) вместе с опорными кривыми $O(n)$ и $O(n^2)$, где n — число правил. Все замеры лежат между этими кривыми, и чем больше доля аномалий, тем ближе к $O(n^2)$.

Это подтверждает заявленную в 4.3 оценку: поиск аномалий с помощью BDD в отличие от попарного сравнения правил снижает сложность анализа набора с $O(n^2)$ до $O(n)$ за счет накопления совместной обработки трафика. Однако для построения отношений между правилами, число аномалий растет быстрее линейного и стремится к $O(n^2)$. На практике, однако, в реальных конфигурациях доля пересекающихся правил невелика, и анализ наборов реалистичного размера укладывается в приемлемое время.

6 Заключение

В данной работе разработана система автоматического анализа и верификации правил фильтрации сетевого трафика, выявляющая ошибки конфигурации межсетевых экранов. Система реализована в виде прототипа *RuleAnalyzer* и интегрирована в систему инвентаризации и валидации сетевых инфраструктур.

Были выполнены следующие задачи:

1. Проведено исследование существующих методов анализа правил фильтрации; выявлено, что известные подходы рассчитаны под конкретную платформу или ограниченный набор полей.
2. Разработана унифицированная вендуро-независимая модель представления правил фильтрации и объектов политики безопасности, охватывающая как классические списки доступа, так и политики межсетевых экранов нового поколения, включая отечественные решения.
3. Выбран и адаптирован для анализа метод формальной верификации FIREMAN.
4. Реализованы алгоритмы обнаружения ошибок конфигурации. Помимо классических аномалий между правилами — затенения, избыточности, корреляции и обобщения — набор обнаруживаемых отклонений расширен: добавлены аномалии отдельного правила (выключенные, истёкшие и неиспользуемые правила) и ошибки уровня объектов политики безопасности. Для правил с невыразимыми в модели квалификаторами применён консервативный анализ через нижнее и верхнее замыкания.
5. Разработан прототип системы анализа правил фильтрации и выполнена его интеграция в систему инвентаризации и валидации сетевых инфраструктур, разрабатываемую в ООО «Дипси Нетворкс».
6. Проведено нагрузочное тестирование, показавшее применимость прототипа к анализу больших наборов правил межсетевых экранов нового поколения.

Планируемые будущие исследования по данной теме:

Инкрементальный анализ. В текущем виде анализатор проверяет конфигурацию, уже развёрнутую на устройстве. Более ценным сценарием является проверка *до* развёртывания (*pre-change analysis*), когда администратор редактирует набор правил и хочет убедиться в отсутствии ошибок перед его применением. Пересчитывать анализ всего набора на каждое изменение нерационально, поэтому необходимо по внесённому изменению определять затронутую им часть набора и пересчитывать только её, сохраняя остальной результат. Это позволило бы встроить проверку прямо в процесс редактирования конфигурации и обнаруживать ошибки на этапе подготовки изменений, а не их эксплуатации.

Анализ достижимости. Разработанный анализатор проверяет *внутреннюю* согласованность набора правил одного устройства. Другой класс задач — анализ достижимости (см. 3.6): определение того, какой трафик может пройти от одного узла сети до другого. В отличие от поиска аномалий, эта задача рассматривает не отдельное устройство, а сеть целиком — последовательность межсетевых экранов на пути трафика с учётом топологии и маршрутизации. Унифицированная модель и аппарат предикатов над пространством заголовков, построенные в работе, применимы и здесь, однако переход к анализу достижимости требует моделирования трансляции адресов (NAT): она изменяет заголовки пакетов между устройствами и в настоящей работе сознательно оставлена за рамками. Развитие анализатора в эту сторону позволило бы перейти от выявления ошибок в наборе правил к проверке сквозной политики доступа в сети.

Список литературы

- [1] Al-Shaer Ehab S and Hamed Hazem H. Firewall policy advisor for anomaly discovery and rule editing // International symposium on integrated network management / Springer. — 2003. — P. 17–30.
- [2] Lihua, Chen Hao, Mai Jianning, Chuah Chen-Nee, Su Zhendong, and Mohapatra Prasant. Fireman: A toolkit for firewall modeling and analysis // 2006 IEEE Symposium on Security and Privacy (S&P'06) / IEEE. — 2006. — P. 15–pp.
- [3] Liu Alex X. Firewall policy change-impact analysis // ACM Transactions on Internet Technology (TOIT). — 2008. — Vol. 11, no. 4. — P. 1–24.
- [4] Gouda Mohamed G and Liu X-YA. Firewall design: Consistency, completeness, and compactness // 24th International Conference on Distributed Computing Systems, 2004. Proceedings. / IEEE. — 2004. — P. 320–327.
- [5] Diekmann Cornelius, Hupel Lars, Michaelis Julius, Haslbeck Maximilian, and Carle Georg. Verified iptables firewall analysis and verification // Journal of automated reasoning. — 2018. — Vol. 61, no. 1. — P. 191–242.
- [6] Wool Avishai. A quantitative study of firewall configuration errors // Computer. — 2004. — Vol. 37, no. 6. — P. 62–67.
- [7] Wool Avishai. Trends in firewall configuration errors: Measuring the holes in swiss cheese // IEEE Internet Computing. — 2010. — Vol. 14, no. 4. — P. 58–65.
- [8] 2010 Data Breach Investigations Report : Rep. / Verizon Business ; executor: Verizon Business RISK Team and United States Secret Service : 2010.
- [9] Hamed Hazem and Al-Shaer Ehab. Taxonomy of conflicts in network security policies // IEEE Communications Magazine. — 2006. — Vol. 44, no. 3. — P. 134–141.
- [10] Ramesh Dhanesh, Harshith RPV, Mahendrakar Sumukh G, Srinivasan Kishore, and Kumar Niharika Prasanna. Exploring contemporary perspectives on the implementation of firewall policies: A comprehensive review of literature // Indiana Journal of Multidisciplinary Research. — 2024. — Vol. 4, no. 3. — P. 218–222.
- [11] Radomskiy S. Security policy rules optimization and its application to the Iptables firewall // Master's Thesis, University of Tampere, Tampere, Finland. — 2011.
- [12] Acharya Subrata, Wang Jia, Ge Zihui, Znati Taieb F, and Greenberg Albert. Traffic-aware firewall optimization strategies // 2006 IEEE International Conference on Communications / IEEE. — 2006. — Vol. 5. — P. 2225–2230.

- [13] Kazemian Peyman, Varghese George, and McKeown Nick. Header space analysis: Static checking for networks // 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12). — 2012. — P. 113–126.
- [14] Zhang Peng, Liu Xu, Yang Hongkun, Kang Ning, Gu Zhengchang, and Li Hao. {APKeep}: Realtime verification for real networks // 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20). — 2020. — P. 241–255.
- [15] Beckett Ryan and Gupta Aarti. Kutra: Realtime verification for multilayer networks // 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). — 2022. — P. 617–634.
- [16] Gouda Mohamed G and Liu Alex X. Structured firewall design // Computer networks. — 2007. — Vol. 51, no. 4. — P. 1106–1120.
- [17] Dhiman Poonam, Saini Neha, Gulzar Yonis, Turaev Sherzod, Kaur Amandeep, Nisa Khair Ul, and Hamid Yasir. A review and comparative analysis of relevant approaches of zero trust network model // Sensors. — 2024. — Vol. 24, no. 4. — P. 1328.
- [18] Golnabi Korosh, Min Richard K, Khan Latifur, and Al-Shaer Ehab. Analysis of firewall policy rules using data mining techniques // 2006 IEEE/IFIP Network Operations and Management Symposium NOMS 2006 / IEEE. — 2006. — P. 305–315.
- [19] Mayer Alain, Wool Avishai, and Ziskind Elisha. Fang: A firewall analysis engine // Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000 / IEEE. — 2000. — P. 177–187.
- [20] Liu Alex X. Formal verification of firewall policies // 2008 IEEE International Conference on Communications / IEEE. — 2008. — P. 1494–1498.
- [21] Zhang Shuyuan, Mahmoud Abdulrahman, Malik Sharad, and Narain Sanjai. Verification and synthesis of firewalls using SAT and QBF // 2012 20th IEEE International Conference on Network Protocols (ICNP) / IEEE. — 2012. — P. 1–6.
- [22] Nelson Timothy, Barratt Christopher, Dougherty Daniel J, Fisler Kathi, and Krishnamurthi Shriram. The Margrave tool for firewall analysis // 24th Large Installation System Administration Conference (LISA 10). — 2010.
- [23] Acharya Hrishikesh B and Gouda Mohamed G. Firewall verification and redundancy checking are equivalent // 2011 Proceedings IEEE INFOCOM / IEEE. — 2011. — P. 2123–2128.
- [24] Kapus Tatjana. Improved formal verification of SDN-based firewalls by using TLA+ // IEEE access. — 2023. — Vol. 11. — P. 107126–107134.

- [25] Mansmann Florian, Göbel Timo, and Cheswick William. Visual analysis of complex firewall configurations // Proceedings of the Ninth International Symposium on Visualization for Cyber Security. — 2012. — P. 1–8.
- [26] Fulp Errin W. Optimization of network firewall policies using directed acyclical graphs // Proceedings of the IEEE Internet management conference / Citeseer. — 2005. — P. 4.
- [27] Hamed Hazem and Al-Shaer Ehab. Dynamic rule-ordering optimization for high-speed firewall filtering // Proceedings of the 2006 ACM Symposium on Information, computer and communications security. — 2006. — P. 332–342.
- [28] Chomsiri Thawatchai, He Xiangjian, Nanda Priyadarsi, and Tan Zhiyuan. Hybrid tree-rule firewall for high speed data transmission // IEEE transactions on cloud computing. — 2016. — Vol. 8, no. 4. — P. 1237–1249.
- [29] Liu Alex X and Gouda Mohamed G. Diverse firewall design // IEEE Transactions on parallel and distributed systems. — 2008. — Vol. 19, no. 9. — P. 1237–1251.
- [30] Bringhenti Daniele, Marchetto Guido, Sisto Riccardo, Valenza Fulvio, and Yusupov Jalolliddin. Automated firewall configuration in virtual networks // IEEE Transactions on Dependable and Secure Computing. — 2022. — Vol. 20, no. 2. — P. 1559–1576.